DISCRETE MATHEMATICS

for Computer Science

Marie Brodsky
Alexander Golovnev
Alexander S. Kulikov
Vladimir V. Podolskii
Alexander Shen

Welcome!

Thank you for downloading this book! It supplements the Introduction to Discrete Mathematics for Computer Science specialization at Coursera and contains many interactive puzzles, autograded quizzes, and code snippets. They are intended to help you to discover important ideas in discrete mathematics on your own, and to show you corresponding applications of these ideas in computer science.

There are 464 problems and 225 code snippets in the book. Most of the problems come with solutions and 219 of them are graded automatically (allowing you to get instant feedback). Please ask questions, report typos, and suggest improvements through this form. This book was last updated on July 27, 2026; get the latest version of the book at https://leanpub.com/discrete-math.

Contents

0. About the Book

0.1 Active Learning

This book covers ideas and concepts in discrete mathematics which are needed in various branches of computer science. To make the learning process more efficient and enjoyable, we use the following *active learning components* implemented through our Introduction to Discrete Mathematics for Computer Science specialization at Coursera.

Interactive puzzles provide you with a fun way to "invent" the key ideas on your own. The puzzles are mobile-friendly, so you can play with them anywhere. The goal of every puzzle is to give you a clean and easy way to state problems where nothing distracts you from inventing a method for solving it. In turn, the corresponding method usually has a wide range of applications to various problems in computer science.

Autograded quizzes allow you to immediately check your understanding after learning a new concept or idea.

Code snippets are helpful in two ways: 1) they show you how ideas from discrete mathematics are used in programming, and 2) they serve as interactive examples and challenges: tweak the given piece of code, run it, and see what happens.

Programming challenges will help you to solidify your understanding. As Donald Knuth said, "I find that I don't understand things unless I try to program them."

0.2 Problem-based Learning

Throughout the book (and the associated specialization at Coursera) we follow a "try this before we explain everything" approach: we always ask you to solve a problem first, and then we explain how to solve it and introduce important ideas needed to solve it. We believe, this way you will get a deeper understanding and also develop a better appreciation for the beauty of the underlying ideas (not to mention the self-confidence that you get if you invent these ideas on your own!). Don't be discouraged if you can't solve all the problems. Just having attempted them is often enough to engage your mind, and make you more curious about the solution.

We use the following two basic types of questions in the book.

Stop and think questions invite you to slow down and contemplate the current material before continuing to the next topic. We *always* provide an answer to the corresponding question right after it. We strongly encourage you (as the name suggests) to stop and do your best to answer the question.

Problems usually require more effort to solve. We use some of them to warm you up and to develop your curiosity. Such problems are followed by detailed solutions. Some other problems are left for you as exercises.

Many questions in the book are *graded automatically* through Coursera. They are marked with:

Try it: Coursera, external.

Both these links are clickable: the first one opens the corresponding autograded puzzle at Coursera (this requires an active subscription to the specialization), the second one opens the corresponding interactive puzzle (and requires no subscription). At the same time, the book is self-contained: if you are unable to watch the videos and access the interactive puzzles at Coursera, just read the book and solve the problems on a piece of paper.

0.3 Python Programming Language

0.3.1 Why Programming?

Why on earth do we start the book with discussing a programming language? After all, this is a math (rather than programming) book!

That's true. But we believe that many pieces of code shown in this book will help you in many ways:

- They will show you a rich variety of applications of discrete math ideas in various branches of computer science.
- Code snippets can serve as interactive examples: you may want to tweak the given piece of code, run it, and see what happens.
- By trying to implement a particular idea, you are forced to understand every single detail of it.
- It is often easier to reason in terms of specific objects in programming rather than abstract mathematical concepts.

We have set up everything in a way that will allow you to run the code snippets used in this book even if you have never tried to write a program before. You don't even need to install or set up anything: everything can be run in the cloud, through your Internet browser. At the same time, we also provide instructions for those who would like to learn the basics of `Python` while learning discrete math.

0.3.2 Why Python?

OK, let's do some programming while learning discrete math. But why `Python` *instead of any other popular programming language?*

Let us convince you that `Python` is an excellent choice for our purposes.

High-level language. It is particularly easy to start using `Python` (even if you haven't programmed before). The syntax is reader friendly (and close to a natural language). The code is compact: most of the pieces of code in this book are less than ten lines long!

Interactive mode. It can be used in an interactive mode (also known as REPL, for read-eval-print loop). This allows you to talk to your computer using `Python` as a language: the computer then *reads* your input, *evaluates* it, and *prints* the result. This way, you work stuff out and get instant feedback from the machine.

"Batteries included". The `Python` standard library offers a wide range of facilities, and many external libraries are available as well. In particular, this will allow us to generate a random sequence, plot a function, and draw a graph in just one line of code!

This (partly) explains why `Python` is often used for software prototyping, and in such areas as machine learning, data science, and web development.

Of course, advantages always come at the cost of some disadvantages. The high-levelness of `Python` makes it less flexible in performance tuning. This is OK for us, as we will only be using simple snippets of code where optimizing is not an issue.

0.3.3 How to Catch Up with Python?

OK, let's try! Where do I start?

Locally. To install Python on your machine, go to the Get Started section of python.org and follow the instructions. If you are new to Python, we encourage you to install PyCharm to start working with Python: this (free of charge) professional IDE will make the process of writing and running your code smoother and more efficient.

In the cloud. Alternatively, you may run all our code snippets from your Internet browser, without installing or configuring anything on your machine. To do this, visit the repository page and click the badge "Open in Colab". This will show you a list of notebooks that can be run in an interactive mode right in your browser (together with links to a tutorial on notebooks).

0.4 Acknowledgments

This book was greatly improved by the efforts of a large number of individuals whom we owe a debt of gratitude.

We thank the students of the Coursera specialization as well as the students of the Modern Software Engineering B.Sc. program at St. Petersburg State University for their continuous and valuable feedback. We also thank Jerry Allen, Huck Bennett, Anuj Kumar Karmakar, Stewart Lewis, and Terence Minerbrook for carefully reading an earlier draft of this book.

We are grateful to Anton Konev and Daria Borisyak for leading the development of interactive puzzles. We thank Vitaliy Polshkov for reviewing our Python code.

Part I
Mathematical Thinking in Computer Science

1. Proofs: Convincing Arguments

Why are some arguments convincing while others are not? What makes an argument convincing? How can you establish your argument in such a way that no room for doubt is left? How can mathematical thinking help us deal with this? In this chapter, we will start by digging into these questions. Our goal here is to learn by examples how to understand proofs, how to discover them on your own, how to explain them, and — last but not least — how to enjoy them: we will see how a small remark or a simple observation can turn a seemingly non-trivial question into one with an obvious answer.

1.1 Warm Up

1.1.1 Why Proofs?

Proofs are absolutely necessary in mathematics, computer science, programming, and many other areas. Once you have an algorithm for a problem, you need to prove that it is correct. "The program works for me, what else do I need?" is not an approach that would scale well. A library function may run billions of times while being used by thousands of diverse programs. If it returns a single wrong result, say, once in every million calls, it could be disastrous. Think of an air traffic controller. If in their entire career they receive information with just one wrong value, just one time, hundreds of people could perish.[1] This is why mathematical proofs must be rigorously demonstrated to provide correct conclusions.

That is why throughout the whole book we will be focusing on formal proofs. Here, "formal" does not mean "long" or "unclear"! We will encounter many short and elegant proofs that are formal and convincing. Using our carefully designed interactive puzzles, we will try to push you softly to discover some of the proofs on your own. Nothing compares to the sense of happiness and self-satisfaction of an "Aha!" moment when you find a solution to a mathematical problem!

[1] For some specific examples, Google for "Bugs in the Space Program", "Therac-25", and "Toyota unintended acceleration".

1.1.2 Tiling a Chessboard

Problem 1 Can a chessboard be tiled by domino tiles? Here, a chessboard is an 8×8 square divided into 64 squares 1×1 (see Figure 1.1), a domino tile is a 1×2 (or 2×1) rectangle, and by saying "tiled" we mean that there are no overlaps or empty spaces. Try it (question 1): Coursera, external.

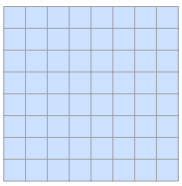


Figure 1.1: An 8×8 chessboard. Can it be tiled with domino tiles?

Yes, there are many such tilings. Two of them are shown in Figure 1.2.

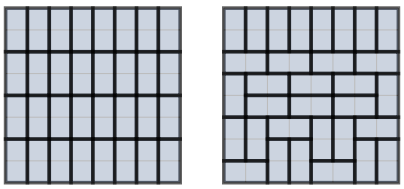


Figure 1.2: Two examples of tiling a chessboard.

Stop and Think! Do we need both these examples to solve Problem 1, or is one enough?

In fact, one example is enough: any such example shows that it is possible to tile a chessboard.

Problem 2 Now consider the chessboard without one of the corners, see Figure 1.3. Can we tile it with domino tiles? Try it (question 2): Coursera, external.

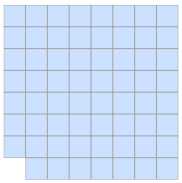


Figure 1.3: A chessboard without a corner. Can it be tiled with domino tiles?

Let's try. For example, let us use horizontal tiles, starting from the top row. Everything goes OK until we come to the bottom row, see Figure 1.4. This last row is problematic. We can put three tiles, but one cell is left uncovered.

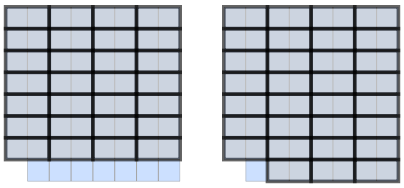


Figure 1.4: An unsuccessful attempt to tile a chessboard without a corner.

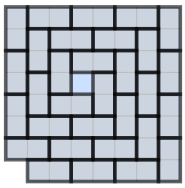


Figure 1.5: An unsuccessful attempt at a spiral tiling of a chessboard without a corner.

Stop and Think! Can we say now that Problem 2 is solved and we proved that the required tiling does not exist?

No, we cannot: one attempt to tile the board was unsuccessful. But this *does not* mean that the task is impossible. We are not limited to using horizontal tiles only; we can try doing something more sophisticated. Let us try a spiral, see Figure 1.5.

Stop and Think! Can we say now that Problem 2 is solved and we have proven that the required tiling does not exist?

Of course not: we tried twice, but there are many more ways to try. What if some of them are successful? For a smaller board we may try all possibilities. Say we invent a systematic way to enumerate them, making sure that no possible tiling has been missed. However, here the space of possibilities is rather large.

Stop and Think! Can you find a tiling or some general reason why we will always fail?

More specifically:

Stop and Think! Imagine there is a tiling of an 8×8 chessboard without a corner, by 1×2 tiles. How many tiles are in this tiling?

The full board contains $8 \times 8 = 64$ cells, so without a corner we have $64 - 1 = 63$ cells. Each domino tile consists of two cells, so the answer is $63/2 = 31.5$ tiles.

Stop and Think! The answer 31.5 is absurd: the number of tiles should be an integer. How is this even possible?

Recall the assumption we started with: "Imagine there is a tiling…". *If* there was a tiling of 63-cell board with domino tiles, it *would* use $63/2 = 31.5$ tiles. This, of course, is impossible — therefore, such a tiling does not exist. Thus, we get the proof we were looking for.

Problem 3 Consider an 8×8 chessboard without two adjacent corners, see Figure 1.6. Can it be tiled by domino tiles? Try it (question 3): Coursera.

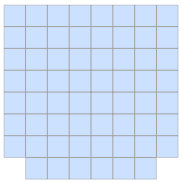


Figure 1.6: A chessboard without two adjacent corners. Can you tile it with domino tiles?

We already know that one should find the number of tiles needed: we have $8 \times 8 - 2 = 62$ cells, so we need $62/2 = 31$ tiles. We got an integer number, so we do not run into a problem and the tiling exists.

Stop and Think! Do you agree with this reasoning?

If you do, you are too fast. The argument shows only that *if* a tiling existed, it *would* consist of 31 tiles. But it does not show that a tiling exists. Informally speaking, we see only that some specific obstacle (non-integer number of tiles) does not prevent the existence of a tiling, but there may be other obstacles.

Stop and Think! Give a correct proof of the existence of a tiling for the board without two adjacent corners.

Here it is, see Figure 1.7.

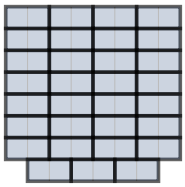


Figure 1.7: A tiling of a chessboard without two corners.

After training with these simple examples, we can tackle a more difficult question.

Problem 4 Consider an 8×8 chessboard without two opposite corners, see Figure 1.8. Can it be tiled by domino tiles? Try it (question 4): Coursera.

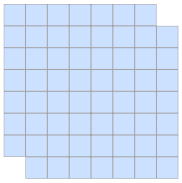


Figure 1.8: A chessboard without two opposite corners.

Again, the board contains $8 \times 8 - 2 = 62$ cells, so $62/2 = 31$ tiles would be needed. This is an integer number. But we already know that it does *not* mean that a tiling exists. Mathematicians would say that the even number of cells is a *necessary* condition for the existence of a tiling, but we do not know whether it is a *sufficient* condition.

Stop and Think! Can you complete the existence proof by constructing some tiling?

Let's try. One such attempt is shown in Figure 1.9. As you see, this attempt was not successful: two cells are not covered.

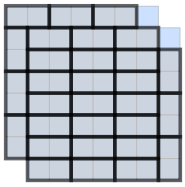


Figure 1.9: An unsuccessful attempt to tile a chessboard without two opposite corners.

The situation does not look hopeless: two cells are not covered, and the only problem is that they are not neighbors, so we cannot cover both by one tile. But maybe one can move some tiles to make the non-covered cells neighboring? Or maybe we can just start anew and have better luck? Let us try, see Figure 1.10. Now the empty cells are in the same column, but they are not neighbors.

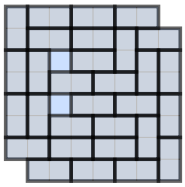


Figure 1.10: Another unsuccessful attempt to tile a board without two opposite corners.

If you play a bit more with this puzzle (level 3), you will see that this is more than just bad luck: every time at least two uncovered cells remain. But why?

Stop and Think! How can we prove that such a tiling is not possible?

Here a new tool is needed. If you are a chess player or have seen a real chessboard, you may have noticed that our drawings ignore one important feature of a chessboard. It has cells of two colors, usually black and white. In our color scheme, we will distinguish light and dark cells, see Figure 1.11.

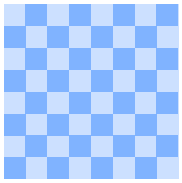


Figure 1.11: Chessboard coloring.

Stop and Think! How many dark and how many light cells are there on the chessboard?

Each row contains 4 light and 4 dark cells, so in total we have $8 \times 4 = 32$ light and 32 dark cells.

Thus, we have the same number of light and dark cells. Could we see this without counting them? One could show that the number of chairs in a room is equal to the number of people in it by asking everyone to sit down: if each person is seated and no chairs are empty, these two numbers are equal. (Unless somebody sits on two chairs or some chair is shared.)

Stop and Think! Can you prove in a similar way that the number of dark cells is the same as the number of light cells, by pairing them into light-dark pairs?

Any tiling of a chessboard will work: looking at the chessboard, we see that neighboring cells are always of different colors, hence every tile is a dark-light pair (Figure 1.12).

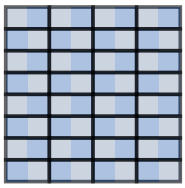


Figure 1.12: Pairing proves that the number of light cells is the same as the number of dark cells.

After this digression let us return to our board without opposite corners (Figure 1.13).

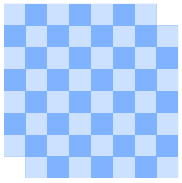


Figure 1.13: Looking again at the board without opposite corners.

Stop and Think! How many dark and how many light cells are on this board?

We do not need to count them again: two corner cells that are deleted are both dark. Thus, we have 30 dark cells and 32 light cells.

Stop and Think! Do you see why this board cannot be tiled by dominos?

Imagine that a tiling existed. It would use $62/2 = 31$ tiles. Each tile covers one dark cell and one light cell. So in total all tiles would cover 31 dark cells and 31 light cells. And — *aha!* — our board has 30 dark and 32 light cells. This mismatch shows that tiling it is impossible.

Let us repeat the argument in a slightly different way. If a board can be tiled, then the numbers of dark and light cells are the same (=the number of tiles), because each tile covers one dark and one light cell. Hence, if these two numbers (dark and light cells) are different, no tiling is possible.

In a more concise exposition, this argument could be compressed into one paragraph.

Theorem 1.1.1 An 8×8 chessboard without two opposite corners cannot be tiled by 1×2 dominoes.

Proof. Consider the standard coloring of the chessboard with two colors (dark and light) where neighboring cells have opposite colors. It is easy to see that

- each tile contains one dark cell and one light cell;
- the board has 32 cells of one color and 30 cells of the other color (two deleted corners have the same color).

The first observation implies that any tileable region has the same number of dark and light cells, and then the second observation shows that our board is not tileable. ■

Stop and Think! We have seen a partial tiling of the board without two opposite corners where two cells remain uncovered. What are the colors of these cells? (Try to give an answer without looking at the picture of the tiling.)

We do not need to know the specific tiling: if we have 32 light and 30 dark cells, and the tiling pairs every light and dark cell with two remaining unpaired, they must be light colored cells.

We have seen that a chessboard with one deleted cell is not tileable for trivial reasons (the number of cells is not even). For two deleted cells we had two examples. The first one, without two neighboring corners, was tileable; the other one, without opposite corners, was not.

Stop and Think! Why can't the argument that we use to prove the non-tileability in the second case be applied to the first case? What is the difference?

Let us formulate this question in a more specific way. Consider a chessboard without two cells. We know that sometimes the rest is tileable (example: two adjacent corners) and sometimes it is not (example: two opposite corners). Try it (questions 5 and 6): Coursera.

Stop and Think! Can you state a general rule that distinguishes between tileable and untileable boards without two cells?

The following problem provides an answer to this question.

Problem 5 Prove that a chessboard without two cells can be tiled by domino tiles if and only if the deleted cells are of opposite colors.

The statement includes a strange expression "if and only if" (also known as "iff"). This mathematical jargon means that we have to prove two things:

- if we delete two cells of the opposite colors, then the rest is tileable ("if" part);
- if the board without two cells is tileable, then the deleted cells are of opposite colors ("only if" part).

Stop and Think! We have shown that any tileable region has the same number of dark and light cells, so the board without two cells of the same color is not tileable. What did we prove: the "if" part or the "only if" part?

It remains to prove that the board without one dark and one light cell is always tileable. To keep the suspense, we will not give the solution of this problem. Here is a diagram to think about, Figure 1.14. If you are old enough (some of the authors are), you may remember the computer game where a growing snake moves along itself eating food items placed in certain cells and increasing its length. In terms of this game, this picture shows a snake that has reached maximal possible length (includes all cells) so its head can be glued to its tail.

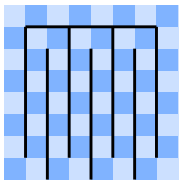


Figure 1.14: This "circular snake" helps us to prove that the board without two cells of opposite colors is tileable. Do you see how? For starters, tile the entire board by cutting the snake into domino tiles. What happens if you delete two cells from the snake?

Problem 6 We want to cover the figure shown in Figure 1.15 by 1×2 domino tiles. Is it possible (a) if we cover the highlighted cell by a horizontal tile? (b) if we cover the highlighted cell by a vertical tile? Try it (questions 1 and 2): Coursera.

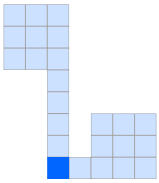


Figure 1.15: Board for Problem 6.

Now consider a 5×5 board divided into 25 cells 1×1 (Figure 1.16). It is not possible to tile it

Figure 1.16: A 5×5 board (Problem 7).

by 1×2 tiles. (Why? Since the number of tiles needed for that, i.e., $25/2 = 12.5$, is not an integer.) However, if we delete one cell, there may be a chance to cover the remaining 24 cells by 12 tiles. For example, if you delete the left upper corner, you can tile the rest using vertical tiles in the first column and horizontal tiles elsewhere. Let us call a cell *good* if the rest (5×5 board without this cell) can be tiled by dominoes.

Problem 7 Which of the cells are good? How many good cells are there? Try it (question 3): Coursera.

Problem 8 Can we tile an 8×8 board by 1×3 tiles? (They can be placed both horizontally and vertically.) Can we tile 8×8 board without one corner by 1×3 tiles?

Problem 9 Can we tile a 10×10 board by 2×2 tiles? Can we tile this board by 1×4 tiles (that can be placed horizontally or vertically)?

Dark and light cells strike back. We have a full characterization of the tileability of a chessboard without two cells: if they are of the same color, then there is no tiling; otherwise, the tiling exists. But what if the board is missing more than two cells? Specifically, do you see a way to implement a program that is given a subset of the cells of the board (i.e., for each cell it is indicated whether it is present or not) and quickly checks whether this region is tileable? We'll learn how to do this later in the book, when we study matchings in graphs! Interestingly, the solution will be based on dark and light cells again. (Technically, one should look for a maximal matching between dark and light cells in the bipartite neighborhood graph; we will explain what all these words mean.)

1.2 Existence Proofs

In this section, we study *existence proofs*, i.e., proofs of *existential statements*. For example,

> There exists an object that satisfies a particular property.

To prove this statement, we can provide an example of such an object: this is called a *constructive proof*.

There is another way to prove an existential statement: we can prove that such an object exists without providing an example of the object! This sounds counterintuitive, doesn't it? These are known as *non-constructive proofs*, and we will see them in Section 1.2.4. For now, we will work with constructive proofs.

1.2.1 One Example Is Enough

We have seen constructive proofs of existential statements in the previous section.

Stop and Think! In the previous section, we proved statements of two types: (a) some region can be tiled by dominoes, and (b) some region cannot be tiled. Which type consists of existential statements: (a) or (b)?

The existence of a tiling is (by definition) an existential statement. We proved it by showing an example of the required tiling. As we have said, one example is enough. This already constitutes a formal proof of existence. One does not need to explain how this example was found, though this may be an interesting and non-trivial question (discussed later in Section 2.1).

Problem 10 Is it possible to cut the figure shown in Figure 1.17 into two congruent pieces, i.e., into two pieces of the same shape and size? (In other words, does there *exist* a way to cut the figure into two congruent pieces?)

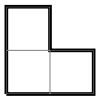


Figure 1.17: Can you cut this figure into two congruent pieces?

For this problem, a solution is not difficult to find, see Figure 1.18.

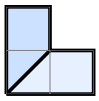


Figure 1.18: A solution to Problem 10.

Stop and Think! What if we want to cut the same figure into three congruent pieces?

This is even easier since the figure consists of three squares. To make things more challenging, what if we pose the same question with four pieces?

Problem 11 Prove that the same figure can be cut into four congruent pieces. (Equivalently, prove that there *exists* a way to cut the figure into four congruent pieces.)

The hint is that the small pieces could be of the same shape as the figure itself. The construction is given in Figure 1.19. Note that this example alone is enough to solve the problem. It provides a complete proof, and we do not need to add anything else.

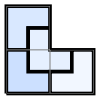


Figure 1.19: A solution to Problem 11.

Let's consider a somewhat more advanced problem of a similar flavor.

Problem 12 Prove that the octagon shown in Figure 1.20 can be cut into two congruent pieces (an octagon is a polygon with eight sides). Try it (question 2): Coursera.

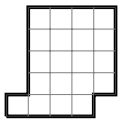


Figure 1.20: Can you cut this octagon into two congruent pieces?

This problem might look difficult at first, but there is again a simple solution. We can actually cut this figure along grid lines, see Figure 1.21. (To see that two pieces are the same, just move the left piece two cells to the right.)

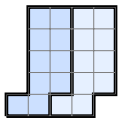


Figure 1.21: A solution to Problem 12.

How curious! There is another solution to this puzzle, see Figure 1.22.

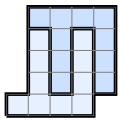


Figure 1.22: Another solution to Problem 12.

Problem 13 Prove that this figure can be cut into *three* congruent pieces.

In this problem, we do *not* require the cuts to go along the grid lines. This allows us to solve the problem by extending the previous solution a bit; see Figure 1.23.

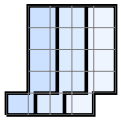


Figure 1.23: A solution to Problem 13.

If we required the cuts in this solution to go along the grid lines, our task would be impossible.

Stop and Think! Do you see why?

The reason is that the figure consists of 20 cells, and 20 is not divisible by 3.

We conclude with a more challenging problem of the same type.

Problem 14 Prove that it is possible to cut the figure from Figure 1.24 into two congruent pieces.

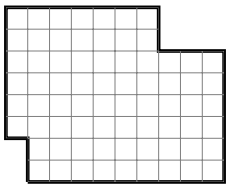


Figure 1.24: Can you cut this figure into two congruent pieces?

Finding a solution to this problem is difficult, so don't spend too much time on it. Knowing that a solution exists but not knowing what it is can be frustrating. For this reason, we provide a solution in Section 7.1.

Let us illustrate the "one example is enough" principle physically rather than mathematically. Imagine that you have three sticks and a length of string. Tie the string around the three sticks so that they form a free-standing structure in which the sticks do not touch. Could you make a free-standing structure with these restrictions? Put another way, does such a structure *exist*? It may surprise you, but the answer is yes! Again, to prove this existential statement, we only have to provide a single working example. See the video in our course or this Wikipedia page.

1.2.2 Existential Statements in Number Theory

Recall that an integer a is *divisible* by a positive integer b if $k = a/b$ is an integer. In other words, a is divisible by b if there *exists* an integer k such that $a = kb$.

In Python, to check whether a is divisible by b, one checks whether the *remainder* of a when divided by b is equal to zero. The remainder is found using the *modulo operator* — `%`. The following snippet shows that 237 is divisible by 3 and is not divisible by 7.

```
print(237 % 3)
print(237 % 7)
```

```
0
6
```

Stop and Think! Is 123 123 123 divisible by 123?

Yes, it is: $123\,123\,123 = 123 \cdot 1\,001\,001$, so we may take $k = 1\,001\,001$ in the definition above. The quotient k can easily be found with a computer program (or just a calculator). You may also get the answer without any tools (even without pencil or paper) if you think about the long division procedure. (Be careful: a common error is to get $k = 111$.)

Problem 15 Is there a positive integer that is divisible by 13 and ends with 15?

To prove that such a number exists, it is enough to give a single example. One such example is 715: it ends with 15 and it is divisible by 13 ($715 = 13 \cdot 55$). This already proves existence, and we don't even need to explain how we found this integer. Still, the following three lines of code help to find all such integers in the range $[0, 9999]$.

```
for n in range(10 ** 4):
    if n % 13 == 0 and n % 100 == 15:
        print(n)
```

```
715
2015
3315
4615
5915
7215
8515
9815
```

This program checks all numbers in the `range(10 ** 4)`. Here, `10 ** 4` stands for $10^4 = 10\,000$. In Python, `range(N)`, where `N` is a non-negative integer, is a *list*[2] (sequence) of `N` numbers $0, 1, 2, \ldots, N-1$. The `for`-loop goes over all of them in this order; the `if` operator checks whether they have the required properties. The last two digits of an integer `n` can be computed as `n % 100`. In general, `n % m` denotes the *remainder* when dividing `n` by `m`.[3]

[2] Technically speaking, in version 3 of Python `range(N)` is no longer a list, but a so-called *generator*, but for us the difference is not so important.

[3] Imagine we have n identical books on the table and pack them into boxes that contain m books each. Then $n \% m$ books *remain* unpacked.

Problem 16 Is there an integer that is divisible by 15 and ends with 13?

In this case, a similar program will produce no output. This does not prove that there is no such integer, since the program checks only the positive integers below 10^4. However, no such integer exists: it would have to be divisible by 5 (since it is divisible by 15), but all integers divisible by 5 end with either 0 or 5.

Problem 17 Find a two-digit (positive) integer that becomes 7 times smaller when its first (=leftmost) digit is removed.

Let's try. Consider all two-digit integers that are divisible by 7:

$$14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84, 91, 98.$$

We know that dividing the required integer by 7 should result in a single single-digit integer. This allows us to rule out all numbers starting from 70 from the list. We can then check manually that out of the remaining numbers the only one satisfying the required property is 35.

The argument above is simple, but still some reasoning is needed. One could use a brute force search instead.

```
for n in range(10, 100):
    if n == 7 * int(str(n)[1:]):
        print(n)
```

```
35
```

This code goes through all integers in the range $[10, 99]$. In general, `range(a, b)`, where `a` and `b` are integers satisfying $\texttt{a} \le \texttt{b}$, denotes a list that contains `a`, `a+1`, ..., `b-1` (and so is empty if $\texttt{a} = \texttt{b}$). To remove the first digit of a number, we convert it to a string (by calling the `str()` function), then use slicing (`[1:]`) to remove the first symbol of the resulting string, and finally convert the resulting string back to an integer.

The expression `int(str(n)[1:])` removes the first digit from *any* positive integer. However, if n is a *two-digit number*, then removing its first digit is the same as leaving its last digit which can be achieved simply by `n % 10`.

```
for n in range(10, 100):
    if n == 7 * (n % 10):
        print(n)
```

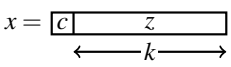
```
35
```

Problem 18 Find an integer that becomes 57 times smaller when its first digit is removed.

Solving this problem on a piece of paper is not easy either, but it is possible to adjust our code above (try it!) and get 7 125. Indeed, $7\,125 = 57 \cdot 125$ (check that this is correct!). Note that you don't need to include the program in the solution: an example is enough. Questions about why and how this number was picked are irrelevant, since this example already answers the question. On the other hand, checking that $7\,125 = 57 \cdot 125$ *is* a part of the solution (even if this part is left to the reader, as we did above).

Stop and Think! Do you see a way to find this example by hand?

Here is one possible line of reasoning. Let c be the first digit of the unknown number x, let z be the number x without the first digit, and let k be the number of digits in z.



Then,

$$x = c\underbrace{00\ldots00}_{k \text{ zeros}} + z = c \cdot 10^k + z.$$

Since x should become 57 times smaller when its first digit is removed,

$$c \cdot 10^k + z = 57 \cdot z.$$

By moving z to the right-hand side, we get

$$c \cdot 10^k = 56 \cdot z. \tag{1.1}$$

Stop and Think! Can you find the value of c by looking at this equation? Recall that c is the first digit, i.e., an integer between 1 and 9.

Note that the right-hand side of (1.1) is divisible by 7. The only case[4] in which the left-hand side is also divisible by 7 is $c = 7$. Plugging this into (1.1), we simplify it to

$$10^k = 8z.$$

Again, this means that 10^k should be divisible by 8. Hence k is certainly greater than 2 as both $10 = 10^1$ and $100 = 10^2$ are not divisible by 8. However, 8 divides 10^3: in this case, $z = 125$. This leads us to the final answer: $x = 7\,125$.

Stop and Think! Are there other examples of a number that becomes 57 times smaller when its first digit is removed?

Note that 10^k is divisible by 8 for *any* $k \geq 3$: indeed, if $k \geq 3$, then 10^k is divisible by 10^3. E.g., for $k = 4$, we get $z = 1\,250$ and $x = 71\,250 = 57 \cdot 1\,250$. This way, we get an infinite family of solutions (for all integers $k \geq 3$). All these solutions are obtained simply by padding our first solution 7 125 by zeros.

Problem 19 Are there other examples besides the solutions we have found for all $k \geq 3$?

Problem 20 Is there an integer that becomes 37 times smaller when its first digit is removed?

Problem 21 Is there an integer that becomes 58 times smaller when its first digit is removed?

Problem 22 Are there positive integers a, b, c such that $a^3 + b^3 = c^3$? Are there positive integers a, b, c, d such that $a^4 + b^4 + c^4 = d^4$?

We will discuss the answer to the last problem later in the book!

In some cases, nobody knows whether a simple existential statement is true or false. For example, nobody knows whether a positive odd perfect integer N exists. Here "odd" means that N is not divisible by 2, and "perfect" means that N is equal to the sum of all its positive divisors, including 1 but excluding N itself (e.g., $N = 28$ is perfect since $1+2+4+7+14 = 28$, but 28 is not odd).

1.2.3 Proofs of Non-existence

A witness can certify that you made a payment to Mr. X or visited Moscow. But it would be much more difficult to prove that you did *not* pay Mr. X or that you have never been to Moscow. The situation with an existential statement is similar: to prove it, one example of an object with the required property is enough. But you cannot disprove it by providing an example or several examples of objects that do not have the required property; some general reasoning is needed to show that objects with the required property do not exist. We have already seen this kind

[4] Here we are cheating a bit: this is not that easy to see. To prove this, one needs some tools from basic number theory that we will discuss later. The left-hand side of (1.1) can be factored as $c \cdot 2^k \cdot 5^k$, and this product is divisible by a prime number 7, so one of the factors should be divisible by 7. The only possibility is $c = 7$. But even if we know nothing about prime numbers and factorization, it would be a natural idea to try $c = 7$; recall that we do not need to explain how the example is found.

of reasoning when proving that some regions are not tileable. In this section, we'll give more examples of this type. (We will give rigorous mathematical notation for existential and universal statements, and for their negations, in Section 4.2.)

Imagine that you go for a hike with a friend and want to split the weight evenly, that is, to split the items you take into two groups of the same weight. Try it: Coursera.

Stop and Think! Suppose we have three items with weights 1, 2, and 3. Can we split them into two groups of the same weight?

This question is almost trivial:

$$1+2=3$$

gives an appropriate split (1 and 2 go in one group, 3 goes in the other one).

Instead of splitting, we could use plus and minus signs to produce a sum of zero:

$$\pm 1 \pm 2 \pm 3 = 0\,.$$

The weights with plus and minus signs form two groups of equal weights. Our example can be represented now as

$$+1+2-3=0 \text{ or } -1-2+3=0\,.$$

Now, let's consider a more interesting example.

Stop and Think! Can we split the weights 1, 2, 3, 4, 5, and 7 into two groups of equal weights? (Reformulation: choose signs in $\pm 1 \pm 2 \pm 3 \pm 4 \pm 5 \pm 7$ to get 0.)

Stop and Think! If we split these weights evenly, what is the weight in each group?

This is easy: the total weight is

$$1+2+3+4+5+7=22,$$

and if we have two equal parts, each one is 11. Hence, we can reformulate the problem as follows: *form a group of weight* 11. Note that it is enough to find one such group: the rest will automatically have the same weight 11.

Stop and Think! Do you see such a group?

Indeed it is easy to find one:

$$4+7=11\,.$$

As we observed, all the remaining weights sum up to the same number:

$$1+2+3+5=11\,.$$

Thus, the problem is solved.

There is no guarantee that an even splitting is always possible. Let us, for example, replace 7 with 6 (one item is now lighter):

Stop and Think! Can we split the weights 1, 2, 3, 4, 5, and 6 into two groups of equal weights?

Let us try the same approach and compute the total weight. Now it is one unit smaller: 21, and each part should have weight $21/2 = 11.5$. This cannot happen since all weights are integers. This means that the desired splitting does not exist (the corresponding existential statement is provably false, as a mathematician would say).

Stop and Think! Can we split the weights 2, 4, 6, 8, 10, and 12 into two groups of equal weights?

This is also impossible. Do you see why? It is essentially the same problem as the previous one, but we have multiplied all the weights by 2.

Alternatively, we can say that the total weight is 42, so if we split the weight into two equal parts, the weight of each part is 21. But all weights are even, so it is impossible to obtain the odd (not divisible by 2) sum of weights.

Stop and Think! Can we split the weights 1, 2, 3, 4, 5, and 17 into two groups of equal weights?

The sum of weights is now

$$1+2+3+4+5+17 = 32,$$

so each part (if a splitting exists) has weight $32/2 = 16$, and this is an integer. Still, the splitting is impossible.

Stop and Think! Do you see why?

The obstacle is easy to see: each part should have weight 16, and one of the items is too heavy (weight 17) for that. We see that the splitting is also impossible, but the obstacle is of a different type.

It would be nice if our list of obstacles was complete. If so, we would be able to check whether the splitting exists by checking whether it is prevented by one of the known obstacles. However, this problem is not nearly so simple. For this problem, no one knows the complete list of obstacles and no one knows a quick way to tell (for a given set of weights) whether the splitting exists.

This problem belongs to the class of so-called NP-*complete problems*. Roughly speaking, this means that no efficient solution is currently known. More precisely, the situation is as follows. We do not know a way to solve this problem (and other NP-complete problems) efficiently, but we also do not know a proof that this is impossible. This question is known as "the P vs. NP problem," and it is one of the most famous and important unsolved mathematical problems (with a $1M prize from Clay Mathematics Institute).

You see that our simple weight-splitting problem is very close to the current boundary of computer science knowledge. Or, it might be better to say that the boundary is very close to it. No current approach works for ten thousand weights that are 64-bit integers (which is not usually a large input for a computer). An exhaustive search in the space of all splittings is not feasible, and no significantly better approaches are known.

1.2.4 Non-constructive Proofs

Can we prove the existence of an object with some property without giving an example? This sounds counterintuitive, but sometimes it is possible.

Stop and Think! There exists an integer n such that

$$2^n + 3^n \leq 10^{1000} \quad \text{and} \quad 2^{n+1} + 3^{n+1} > 10^{1000}.$$

Do you see why?

For small n (say, for $n = 1$), the first inequality is true. On the other hand, for large n, the second one is true. Consider, for example, $n = 4000$. For this n, the first term alone is large enough: $2^{n+1} > 2^{4000} = 16^{1000} > 10^{1000}$. But we need to find n such that *both* inequalities are true at the same time. We need to find n such that $2^n + 3^n \leq 10^{1000}$ but increasing n by 1 reverses the inequality.

Stop and Think! Do you see why there exists an n with this property?

Intuitively, if we were inside and now are outside, we had to cross the boundary at some point. Let us increase n (going from 1 to 4000) and just stop before $2^n + 3^n$ becomes greater than 10^{1000}. This will give us the required value of n. This finishes the proof, but we can also find the value of n as follows.

```
for n in range(4000):
    if 2 ** n + 3 ** n > 10 ** 1000:
        print(n - 1)
        break
```

```
2095
```

Another classical example deals with irrational numbers. Recall that a real number x is called *rational* if it can be represented as $\frac{a}{b}$, where a and b are integers with $b \neq 0$. Otherwise, it is called *irrational*. For example, $\sqrt{2}$ is irrational. Indeed, assume that $\sqrt{2} = \frac{a}{b}$ for some integers a, b. Then, $2 = \frac{a^2}{b^2}$, and hence $2b^2 = a^2$. We may assume that at least one of a and b is odd (if both are even, keep canceling the factor 2 until one becomes odd). If a is odd, then a^2 is odd and cannot be equal to $2b^2$. Therefore, a is even and b is odd (due to our assumption). But this is also impossible: if $a = 2k$, then $2b^2 = a^2 = 4k^2$, so $b^2 = 2k^2$, but b^2 should be odd for odd b. The contradiction shows that $\sqrt{2}$ is irrational.

Problem 23 Prove that there exist two irrational numbers x and y such that x^y is rational.

To prove this, consider $x = y = \sqrt{2}$. If x^y is rational, then we are done. Otherwise, $x^y = (\sqrt{2})^{\sqrt{2}}$ is irrational. But then, raising it to the $\sqrt{2}$ power gives a rational number:

$$\left((\sqrt{2})^{\sqrt{2}}\right)^{\sqrt{2}} = (\sqrt{2})^{\sqrt{2}\cdot\sqrt{2}} = (\sqrt{2})^2 = 2.$$

Note that we again proved that there exist x and y satisfying the requirements of Problem 23 without explicitly specifying the values of x and y!

Problem 24 Prove that either $\sin^2(10^{1000}) \geq 1/2$ or $\cos^2(10^{1000}) \geq 1/2$. [Hint: check out basic trigonometric identities or the Pythagorean theorem.]

It is not so easy to determine which of these two inequalities holds. Indeed, the first step in computing these two values would be to reduce 10^{1000} radians to the interval $[0, 2\pi]$. To do this, one needs to know the value of π with very high precision (about a thousand decimal digits). Still, we know for sure that one of these values is guaranteed to be at least $1/2$, even though we do not know which one.

The previous two examples are based on the so-called *law of excluded middle* that says that, for any statement, either it is true or its negation is true. For example, for any number x, either $x = 5$ or $x \neq 5$. We do not need to know the value of x to be sure that one of these two statements is true, and at the same time without knowing the value of x we do not know which of these two statements is actually true.

Figure 1.25: There are ten pigeons and nine holes; therefore, there must be a hole having more than one pigeon. (Source: Wikipedia.)

Another useful method for proving existence non-constructively is *the pigeonhole principle*: if there are more pigeons than holes, and each pigeon occupies some hole, then two pigeons share the same hole (see Figure 1.25). As simple as it is, it has many non-trivial applications.

Problem 25 Prove that there exists a positive integer divisible by 57 that uses only digits 0 and 1.

Consider the numbers $1, 11, 111, 1111, 11111, \ldots$ and divide each of these numbers by 57. We get a sequence of remainders: $1 \bmod 57 = 1$, $11 \bmod 57 = 11$, $111 \bmod 57 = 54$, $1111 \bmod 57 = 28$, $11111 \bmod 57 = 53$ and so on ($a \bmod 57$ is the remainder of a when divided by 57). Since this sequence is infinite and there are only 57 possible remainders modulo 57, at some point we should see repetitions, i.e.,

$$\underbrace{111\ldots111}_{m \text{ digits}} \bmod 57 = \underbrace{111\ldots111}_{n \text{ digits}} \bmod 57$$

for $m \neq n$.

Stop and Think! Do you see how this helps?

If two numbers have the same remainder when divided by 57, then their difference is a multiple of 57. But the difference has the decimal representation $11\ldots1100\ldots00$, so it is the desired example. (Assuming $m > n$, we have n zeros at the end and $m - n$ ones before them.)

In fact, one can find a number of the form $111\ldots111$ that is divisible by 57; the trailing zeros are not needed. We will see later in the book that trailing zeros do not help either, since 10 is coprime with 57 (i.e., the only positive number that divides 10 and 57 is 1).

Problem 26 Using a `Python` program (if needed), find a number of the form $111\ldots111$ that is divisible by 57.

Problem 27 As of December 2020, there are 66 758 learners enrolled in our Mathematical Thinking in Computer Science course. Prove that there are 183 of them who share a birthday.

Often a non-constructive proof can be replaced by a constructive one. Sometimes a brute force search helps; sometimes another argument is needed. For Problem 23, one may note that

$$(\sqrt{2})^{2\log_2 3} = 2^{\log_2 3} = 3$$

and both $\sqrt{2}$ and $2\log_2 3$ are irrational (and 3 is rational).

Problem 28 We discussed why $\sqrt{2}$ is irrational above. Show that $2\log_2 3$ is also irrational. [Hint: unique factoring is useful here.]

One may ask whether $\sqrt{2}^{\sqrt{2}}$ is rational or irrational. The Gelfond–Schneider theorem guarantees that this number is irrational (and even *transcendental*, which means that it is not a root of a non-zero polynomial with integer coefficients).

We end this section with two examples where finding a constructive proof is difficult. Let us generate 30 seven-digit numbers.

```
from random import randint, seed

seed(14)

for i in range(30):
    print(randint(10 ** 6, 10 ** 7 - 1), end=' ')
    if i % 6 == 5:
        print()
```

```
2792285 9843470 5142886 5548558 5291201 5882438
2218459 8545410 6083294 8828556 7655079 7607029
2986415 5421072 4745783 6295863 7006791 5368826
7051040 9661551 3511608 3701978 5619271 3767372
1177927 2173344 3064039 6655032 1466196 2395998
```

The function `randint(a,b)` returns a random integer in the range `a...b`. We call it 30 times, using some additional tricks to produce a nice printout. The second argument `end=' '` tells the `print` function to print a space character (instead of the newline character it prints by default). Then a call to `print()` is used to produce a newline character after each group of six numbers. (The call to `seed()` initializes the pseudorandom generator. This is done for reproducibility: it ensures that every time this program is run, the same 30 integers are generated. To get 30 different numbers, just change the argument of the `seed` function.)

Problem 29 Prove that there exist two disjoint subsets of this set of 30 integers that have equal sums. (In other words, we can color some of the elements blue, and color some other elements red so that the sum of the red numbers is equal to the sum of the blue numbers.)

It looks counterintuitive: the numbers are rather long, and having two equal sums initially looks like a strange coincidence. Still, the following non-constructive argument proves our claim. For every subset A of this 30-element set, consider an integer $S(A)$, the sum of all elements in A. All $S(A)$ are less than $30 \cdot 10^7$ (about a third of a billion). On the other hand, we have 2^{30} subsets (each can be described by a 30-bit string saying which of the numbers are included and which are not, and there are 2^{30} binary strings of length 30). Now comes the crucial point (the pigeonhole principle):

$$\textit{since } 2^{30} = 1024^3 > 10^9 > 30 \cdot 10^7\textit{, there exist distinct } A \textit{ and } B \textit{ such that } S(A) = S(B).$$

These A and B may include some common numbers, but these numbers can be deleted and still $S(A) = S(B)$ (since we delete the same numbers). This is exactly what we claimed. (Note that A and B are both non-empty: since $A \neq B$, at least one of them is non-empty and has positive sum, and the other has the same sum.) Still, this non-constructive argument does not give any information about these two subsets.

Later in the book, we present a simple `Python` program that finds two such subsets using ILP solvers.

Our last example is *factorization*: one can prove using Fermat's little theorem[5] that the number

412023436986659543855531365332575948179811699844327982845455626433876445565248426198098870423161841879261420247188869492560931776375033421130982397485150944909106910269861031862704114880869705649029036536588674337317208131041051908642547932826013912576240339463732693 91

(it does not fit in a single line!) is *composite*, which means that it is a product of two smaller integers. This is an existential statement (there *exists* a factorization), but nobody has been able to find one. (There was even a $70 000 reward for doing this.)

Another common (and powerful!) way of proving something non-constructively is called the *probabilistic method*. Roughly, its main idea is the following: to prove that an object satisfying a particular property exists, take a *random* object and show that there is a *non-zero* chance that it satisfies the required property. From this, we conclude that an object satisfying the property *exists*. We will see applications of this method later in the book.

Summary

- A chessboard without two cells can be tiled by domino tiles if and only if the deleted cells are of opposite colors.
- The structure of a proof reflects the structure of the claim.
- One example suffices to prove an existential statement.
- A general argument is required to prove that an existential statement is false.
- Non-constructive proofs show that a certain object exists without giving an example.

[5] Fermat's little theorem states that $(a^p - a)$ is divisible by p for any integer a and any prime p. We will discuss this theorem later.

2. Finding an Example

How can we know that an object with certain properties exists? In Section 1.2, we saw that it suffices to give an example of such an object, but finding an example might be a hard problem. One way to find an example is to go through all objects and check whether at least one of them meets the requirements. However, in many cases, the search space is enormous. A computer may help, but reasoning that *narrows* the search space is important both for computer searches and for work done "by hand." In this chapter, we will learn various techniques for showing that an object exists and that an object is optimal among all objects (say, the smallest or largest object that meets the requirements).

2.1 How to Find an Example

We start our journey with the famous engraving "Melencolia I" by Albrecht Dürer! Note the 4×4 square in the upper right corner of the engraving containing all the numbers from 1 to 16 (see Figure 2.1). If you add the numbers in any row, column, or diagonal, you will find that all the sums are equal. Such a square is called a "magic square."

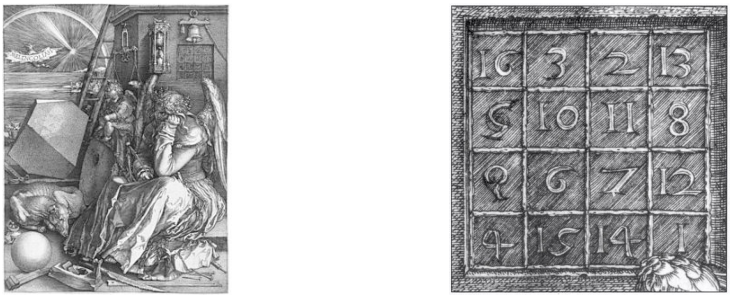


Figure 2.1: Melencolia I by Albrecht Dürer and the magic square in its upper right corner. (Source: Wikipedia.)

Stop and Think! — Recover Dürer's engraving. Some parts of Dürer's engraving are unclear. Can you use the magic square property to recover the first number in the second row? The first number in the third row?

We know that the sum in each row is the same, but *what* is this sum? Since all numbers from 1 to 16 appear in the table once, we know that the sum of all entries in the table is $1+2+\cdots+16=136$. On the other hand, each row has the same sum, and there are 4 rows in the table. This tells us that the sum in each row is exactly 34. Hence, the first number in the second row is 5, and the first number in the third row is 9.

In general, an $n \times n$ magic square is a square in which each integer from 1 to n^2 appears exactly once, and the sums of the numbers in each row, each column, and both diagonals are the same. For what values of n do $n \times n$ magic squares even *exist*? From Dürer's engraving, we know that 4×4 magic squares exist. What about $n \times n$ magic squares for other values of n?

When $n = 1$, the problem is trivial. Namely, we have a 1×1 table with the number 1 written in its only cell. Of course, all sums are the same in this table.

Stop and Think! — 2×2 magic squares. Do 2×2 magic squares exist?

No, there are no 2×2 magic squares! Let us prove this. Consider a 2×2 table with numbers a, b, c, d:

a	b
c	d

To satisfy the magic square requirement, the sums in the first row and in the first column must be the same: $a+b=a+c$, i.e., $b=c$, which contradicts our assumption that all numbers in the table are distinct.

Problem 30 — 3×3 magic squares. Do 3×3 magic squares exist? Try it: Coursera.

Since there are too many different 3×3 tables to check them all, we will first narrow the search space.

2.1.1 Narrowing the Search Space

We want to find a 3×3 magic square. Should we just try all 3×3 tables with numbers from 1 to 9?

Stop and Think! — Number of 3×3 tables. How many 3×3 tables with all numbers from 1 to 9 are there?

There are 9 choices for which number will go in the first cell, 8 choices for which number will go in the second cell, 7 choices for which number will go in the third cell, and so on. This tells us that the total number of such tables is $9! = 1 \times 2 \times \cdots \times 9 = 362\,880$. The exclamation point here denotes the "factorial" of 9, i.e., the product of all integers from 1 to 9. We will discuss this in more detail and see more examples of such calculations later in the book.

While a computer can handle 362 880 cases, there are far too many for us to handle. Let us try to decrease the number of cases we need to consider. Recall that the sums in each row, column, and diagonal are the same. Let us denote this sum by S.

Stop and Think! — Sum in each row. What is the value of S—the sum in each row, column, and diagonal?

The total sum of all entries in the table is $1+2+\cdots+9=45$. The sum of the three rows of the table, i.e., $3S$, is also 45. Thus, $S=15$. In fact, this simple observation already narrows the search space considerably.

Stop and Think! If we know the four numbers in the upper left corner of a magic square (see Figure 2.2), can we compute all the other numbers in the table?

a_1	a_2	a_3
a_4	a_5	a_6
a_7	a_8	a_9

Figure 2.2: Given the four highlighted entries in the upper left corner of a magic square, we can compute the values of the remaining entries.

We know that the sum $S = 15$ in each row, column, and diagonal. Hence, given the numbers a_1, a_2, a_4, a_5, we can compute all the remaining numbers a_3, a_6, a_7, a_8, and a_9. Now, we only need to find the numbers a_1, a_2, a_4, and a_5.

Stop and Think! How many choices are there for the values of a_1, a_2, a_4, and a_5?

Let us count the number of choices: there are 9 choices for a_1; then for each value of a_1, there are 8 choices for a_2; then 7 choices for a_4; finally, 6 choices for a_5. This gives us $9 \times 8 \times 7 \times 6 = 3024$ choices. We do not need to consider all 362 880 tables anymore, because we have reduced the size of the search space by a factor of 120: now it suffices to check only 3024 tables. Let us not rush to check all those tables, but rather try to eliminate more cases.

Problem 31 — Value of the central entry. There is only one possible value of the central element a_5 in a 3×3 magic square. Can you find this value?

Consider the sums of all entries along the four lines shown in Figure 2.3. On the one hand,

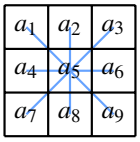


Figure 2.3: Sums of the numbers along the four highlighted lines contain each entry of the table once, but the central entry a_5 four times.

these four sums together should give $4S = 60$. On the other hand, they include each entry a_i once, but a_5 is counted four times. This gives us:

$$4S = 3S + 3a_5 .$$

We conclude that $a_5 = S/3 = 15/3 = 5$. Now we know the value of the central entry in every 3×3 magic square: it is always 5, and we have only $9 \times 8 \times 7 = 504$ tables remaining!

Now, we will see where one can place the number 1 in the table. Since our table is symmetric, there are only two possibilities: either 1 is in a corner, or 1 is in the middle of one of the sides. We consider these two cases.

Case I. 1 is in a corner. For example, let $a_1 = 1$; see Figure 2.4.

From $1 + 5 + a_9 = 15$, we have that $a_9 = 9$. Also, $a_2 + a_3 = a_4 + a_7 = 14$. Can we find distinct numbers a_2, a_3, a_4, a_7 from 1 to 9 satisfying this equation? Yes, there are two ways to get the sum 14: $9 + 5 = 8 + 6 = 14$. But we have already placed 9 at the bottom right corner. This means that there is no way to satisfy the magic square requirement in the first column and the first row simultaneously. We conclude that 1 cannot be placed in a corner.

Case II. 1 is in the middle of a side. For example, $a_2 = 1$; see Figure 2.5.

By inspecting the middle column, we see that a_8 must equal 9. Again, we have that $a_1 + a_3 = 14$, and we already know that this implies that one of these numbers is 8, and the other one is 6.

1	a_2	a_3
a_4	5	a_6
a_7	a_8	a_9

Figure 2.4: One is placed in a corner: $a_1 = 1$. In this case, $a_9 = 9$, and it is impossible to simultaneously satisfy $a_2 + a_3 = a_4 + a_7 = 14$.

a_1	1	a_3
a_4	5	a_6
a_7	a_8	a_9

Figure 2.5: One is placed in the middle of a row: $a_2 = 1$.

Since a_1 and a_3 are symmetric so far, without loss of generality we can assume that $a_1 = 8$ and $a_3 = 6$ (see Figure 2.6).

8	1	6
a_4	5	a_6
a_7	9	a_9

Figure 2.6: One is placed in the middle of a row: $a_2 = 1$. We can compute the values of a_1, a_3, a_8.

Actually, we already have enough information about this magic square to uniquely identify all the remaining numbers in the table. From one of the diagonals, we have that $8 + 5 + a_9 = 15$, so $a_9 = 2$. Similarly, $a_7 = 4$. Then $a_4 = 3$ and $a_6 = 7$ (see Figure 2.7).

It only remains to verify that we have used each number exactly once and that all rows, columns, and diagonals sum to 15. This concludes our proof of the existence of 3×3 magic squares. Note that we reduced the number of cases we needed to consider from 362 880 to two!

We have proved that there exist 3×3 magic squares. Actually, it is possible to show that for every $n > 2$, there is at least one magic square of size $n \times n$. Below, we will see other techniques that often come in handy when trying to find examples.

2.1.2 Using Known Results

We have constructed magic squares, but what can we say about their *multiplicative* versions? Let us say that a 3×3 table with integers is a multiplicative magic square if the *products* of the numbers in each row, column, and diagonal are the same.

Stop and Think! Is there a multiplicative magic square that contains each of the numbers from 1 to 9 exactly once?

No, there are no such squares. Indeed, wherever we place the number 5, some lines will contain 5 and others will not. The products along the former lines will be multiples of 5, while the products along the latter lines will not be multiples of 5, which contradicts the multiplicative magic square requirement. (We explain this divisibility issue in much greater detail later in the book.)

Then, let us try to find magic squares with arbitrary integers. Is this easy? Yes, we can just write the same number in every cell of the table. This finally leads us to an interesting problem.

8	1	6
3	5	7
4	9	2

Figure 2.7: One is placed in the middle of a row: $a_2 = 1$. We can compute all other entries.

Problem 32 Does there exist a multiplicative 3×3 magic square with distinct integers without the restriction that they must be the numbers 1 through 9?

Should we repeat the search from Section 2.1.1? There we had to use numbers from 1 to 9, whereas here we do not have such a restriction. Thus, the search space in our current problem appears to be infinite. As the section title suggests, we should try to use what we have learned so far. Can we take a 3×3 magic square and transform it into a multiplicative magic square? Is there an operation that turns sums into products? Yes, we can use exponentiation! Namely, pick your favorite integer $c > 1$, e.g., $c = 2$, and notice that for every x, y:

$$c^x \cdot c^y = c^{x+y}.$$

Stop and Think! Do you see how to modify the magic square from Figure 2.7 into a multiplicative magic square?

We can take a magic square from the previous section and replace every number x in it with 2^x (see Figure 2.8). Previously, if three numbers x, y, z formed a line, then $x + y + z = 15$. In the new table with numbers $2^x, 2^y, 2^z$, their product has the corresponding property:

$$2^x \cdot 2^y \cdot 2^z = 2^{x+y+z} = 2^{15}.$$

8	1	6
3	5	7
4	9	2

(a)

2^8	2^1	2^6
2^3	2^5	2^7
2^4	2^9	2^2

(b)

256	2	64
8	32	128
16	512	4

(c)

Figure 2.8: (a) "Additive" magic square from Figure 2.7; (b) The corresponding multiplicative magic square; (c) The corresponding multiplicative magic square where we have computed the values of all entries.

This gives us a multiplicative magic square. Indeed, the product in each row, column, and diagonal is 2^{15}, and all numbers in the table are distinct integers. Note that we did not need to go through many cases: we just showed how to use a magic square to build a multiplicative magic square.

Problem 33 The largest number in our multiplicative magic square (Figure 2.8) is 512. Do you see a way to build a multiplicative magic square with distinct positive integers whose largest number is less than 300?

Notice that all numbers in Figure 2.8 are even, so we can divide all numbers in the table by 2 while preserving the multiplicative magic property (see Figure 2.9). (In other words, we can use the numbers from 2^0 to 2^8 instead of 2^1 to 2^9.)

128	1	32
4	16	64
8	256	2

Figure 2.9: Another multiplicative magic square.

There is a construction of a multiplicative magic square in which all numbers are smaller than 40. We start with two distinct "additive" magic squares with very small but not necessarily distinct entries (Figure 2.10 (a) and (c)). We transform one of them into a multiplicative magic square (whose entries are not necessarily distinct) by raising $c = 2$ to the corresponding powers (Figure 2.10 (b)). Similarly, we transform the other one by raising $c = 3$ to the corresponding powers (Figure 2.10 (d)). In Figure 2.10 (b) and (d), we have two multiplicative magic squares with very small entries, but alas, the entries are not distinct. How can we combine two multiplicative magic squares into a new one? We can take two squares and multiply entries located in the same positions to get another multiplicative square. It is easy to verify that the final multiplicative magic square (Figure 2.10 (e)) has 9 distinct numbers, each of which is less than 40. Here it is important that even though each of the squares in (a) and (c) has some duplicate values, the duplicates do not occur in the same locations in both (a) and (c).

1	2	0
0	1	2
2	0	1

(a)

2	4	1
1	2	4
4	1	2

(b)

0	2	1
2	1	0
1	0	2

(c)

1	9	3
9	3	1
3	1	9

(d)

2	36	3
9	6	4
12	1	18

(e)

Figure 2.10: (a) An "additive" magic square with repetitions; (b) a multiplicative magic square with repetitions, constructed by raising $c = 2$ to the powers that appear in the additive version from (a); (c) another "additive" magic square with repetitions; (d) a multiplicative magic square with repetitions, constructed by raising $c = 3$ to the powers that appear in the additive version from (c); (e) the product of (b) and (d) — a multiplicative magic square with small distinct entries.

2.1.3 Identify More Properties

It often helps to identify more properties of an object we are looking for and to use these properties to narrow the search space.

Problem 34 Does there exist a six-digit integer that starts with 100 and is a multiple of 9 127?

We are looking for a number that starts with 100, has three more digits (from 000 to 999), and is divisible by 9 127. This gives us 1 000 candidate numbers, and we can simply check whether one of them is a multiple of 9 127. A computer can easily do this.

```
for i in range(100000, 101000):
    if i % 9127 == 0:
        print(i)
```

```
100397
```

On the other hand, we can find the answer without a computer. Recall that our number must be a multiple of 9 127. This important property significantly reduces the search space: instead of checking all six-digit numbers starting with 100, let us consider only multiples of 9 127. We want to find an integer k such that

$$100\,000 \leq 9127k \leq 100\,999\,.$$

For this, we just need to divide both 100 000 and 100 999 by 9 127:

$$10.95\ldots \leq k \leq 11.06\ldots\,.$$

Thus, we take $k = 11$, and the answer to the problem is $9\,127k = 100\,397$. Our reasoning also shows that 100 397 is the only number satisfying the given requirements.

Problem 35 Does there exist a three-digit integer that has remainder 1 when divided by $2,3,4,5,6,7$?

Again, we can write a simple program that checks all three-digit numbers.

```
for n in range(100, 1000):
    if all(n % m == 1 for m in range(2, 8)):
        print(n)
```

```
421
841
```

Instead, we could easily find such numbers by identifying other properties they have. If N has remainder 1 when divided by $2,3,4,5,6$, and 7, what can we say about the number $N-1$? $N-1$ must be a multiple of $2,3,4,5,6$, and 7. That is, $N-1$ must be a multiple of the *least common multiple* of $2,3,4,5,6$, and 7, which is $4\times 3\times 5\times 7 = 420$. (The least common multiple of a set of numbers is the smallest positive integer that is divisible by all of them — this is a topic we will cover in great detail later in the book, in Section 22.2.1.) Thus, $N-1 = 420k$ for an integer k, and $N = 420k+1$. We do not take $k = 0$ or $k > 2$, because we are looking for a three-digit number N. On the other hand, both $k = 1$ and $k = 2$ give us examples of the desired object: $N = 421$ and $N = 841$.

Problem 36 Does there exist an integer n such that n^2 starts with 31415?

The search space for this problem is seemingly infinite: there are infinitely many integers. Can we identify useful properties of n that would simplify the search? We have two cases: the number of remaining digits is odd or even. We will demonstrate the process for an even number of remaining digits and let you do the same for the odd case in Problem 37. Assume that n^2 starts with 31415 followed by $2k$ digits:

$$n^2 = 31415d_1d_2\ldots d_{2k}\,.$$

Let us look at the square of $(n/10)$. Since $(n/10)^2 = n^2/100$, this number is just n^2 with a decimal point preceding the last two digits:

$$(n/10)^2 = 31415d_1d_2\ldots d_{2k-2}.d_{2k-1}d_{2k}\,.$$

Now instead of dividing n by 10, let us divide it by 10^k.

$$(n/10^k)^2 = 31415.d_1d_2\ldots d_{2k}\,.$$

This already gives us the first digits of n. Indeed, taking the square root of both sides of the last equation and using $\sqrt{31415} = 177.242771\ldots$ and $\sqrt{31416} = 177.245592\ldots$, we obtain

$$177.242771 < (n/10^k) < 177.245593\,.$$

In particular, we can take $n/10^k = 177.243$, i.e., $n = 177243$ and $k = 3$. This gives us an example of such a number, because $n^2 = 31415081049$.

Problem 37 Can you find an integer n starting with 5 such that n^2 starts with 31415?

In order to solve this problem, we suggest repeating our solution, this time assuming that n^2 starts with 31415 followed by an *odd* number of digits.

2.1.4 Solve a Smaller Problem

Problem 38 Alice and Bob live in a country where only coins with values of 7 and 13 florins are used. Luckily, Alice and Bob have infinitely many coins of each type. Find all integer values c such that Alice can pay Bob c florins.

Let us see an example first. If Alice wants to pay Bob 6 florins, she can give Bob a 13-florin coin, and get a 7-florin coin back. Thus, if $c = 6$, Alice can pay Bob c florins.

Stop and Think! How can we check all values of c?

In such cases, a very useful technique is to identify important special cases.

Stop and Think! Is there a value of c such that if Alice can give c florins to Bob, then Alice can pay Bob *any* amount of money?

If Alice can pay $c = 1$ florin, then she can pay any other amount, too! It is not difficult to pay one florin. We already know that Bob can pay Alice 6 florins. After this, Alice gives one 7-florin coin to Bob, so in total Alice has paid Bob one florin. If we repeat this procedure c times, then Alice pays Bob c florins.

By identifying the most important special case ($c = 1$) and finding an example for this case, we solve the problem for all values of c!

Problem 39 Do both Alice and Bob need both kinds of coins? In other words, can Alice pay Bob c florins if Alice has only 7-florin coins and Bob has only 13-florin coins? Try it: Coursera.

Consider another version of this problem.

Problem 40 Alice and Bob have infinitely many coins with values of 15 and 21 florins. Find all values c such that Alice can pay Bob c florins.

Can Alice pay Bob $c = 1$ florin? No matter how many coins with values of 15 and 21 they use, the amount that Alice pays Bob is always a multiple of three. Indeed, 15 and 21 are multiples of three, so any combination of these coins is also a multiple of three. This means that if c is not divisible by 3, then Alice *cannot* pay c florins.

Stop and Think! If c is divisible by 3, can Alice pay c florins?

Again, we do not need to check all such values of c; it is enough to check just one "main" value: if Alice can pay 3 florins, then she can pay any multiple of 3, too. We need only one example for $c = 3$:

$$3 = (3 \times 15) - (2 \times 21)$$

This solves the problem. Alice can pay c florins *if and only if* c is a multiple of 3. Indeed, *if* c is a multiple of 3, then Alice can pay c florins by repeatedly paying 3 florins. On the other hand, Alice can pay c florins *only if* c is a multiple of 3 (recall that any combination of 15 and 21 is a multiple of 3).

Problem 41 We have coupons for 10 free nights at the hotel A, 15 free nights at the hotel B, and 20 free nights at the hotel C. However, there is a restriction: one cannot spend two nights in a row at the same hotel. Can we use these coupons and stay at the hotels A, B, and C for $10 + 15 + 20 = 45$ consecutive nights? Try it (question 1): Coursera.

One way to solve this problem is to identify a smaller and simpler sub-problem. Solving it will lead us to a solution to the original, larger problem. 10, 15, and 20 are multiples of 5, and if we divide all these numbers by 5, then we get a smaller version of this problem.

Stop and Think! Now we have 2 coupons for A, 3 coupons for B, and 4 coupons for C. Say we find a solution for this smaller problem, couldn't we just join five copies of this solution to resolve our larger, original problem?

Almost: we want the first and the last hotels in our small solution to be different, so that when we glue them together we do not spend two consecutive nights at the same hotel.

Hence, we only need to find an example of a schedule in which one spends 2, 3, and 4 nights at the hotels A, B, and C and in which the first and the last hotels are distinct. This small problem is easy to solve: since C has the greatest number of coupons, let us start there and plan to spend every other night at C:

$$\cdot C \cdot C \cdot C \cdot C \cdot$$

Let us add As and Bs in between:

$$A\,C\,A\,C\,B\,C\,B\,C\,B$$

We have solved the smaller problem, and we can concatenate five copies of this solution to solve the original problem.

Problem 42 You have coupons for 10 free nights at the hotel A, 15 free nights at the hotel B, and 30 free nights at the hotel C. However, there is a restriction: you cannot spend two nights in a row at the same hotel. Can you use these coupons and stay at the hotels A, B, and C for $10 + 15 + 30 = 55$ consecutive nights? Try it (question 2): Coursera.

Note that even if you spend every other night at the hotel C, you still need to fill 29 gaps between those nights. We cannot do this because we only have 25 coupons for hotels A and B. Therefore, it is impossible to use your coupons for 55 consecutive nights. We identified an obstacle and proved that there is no schedule for 55 nights; thus, there is no use in trying to find an example.

Summary

- In order to prove that an object exists, it is enough to find one example of such an object.
- Check if you see any obstacles to the existence of such objects.
- Try to narrow the search space.
- Check whether known results let you simplify the problem.
- Identify more properties of the desired object to narrow the search space.
- Identify smaller problems that can lead to a solution to the original problem.

Try to use these techniques to solve the following problems. Try it: Coursera.

Problem 43 Does there exist a 5-digit integer N such that N^2 starts with 27182?

Problem 44 You have 25 coupons for the hotel A, and 20 coupons for the hotel B. What is the maximum number of coupons for the hotel C that you can use if you are not allowed to spend two consecutive nights at the same hotel?

Problem 45 There are fewer than 100 books on a shelf. If you pack them into boxes that contain 3 books each, then 2 books remain on the shelf. If you pack them into boxes that contain 4 books each, then 3 books remain on the shelf. Finally, if you pack them into boxes that contain 5 books each, then 4 books remain. How many books are there on the shelf?

Problem 46 A country uses only coins with values of 12, 20, and 30 florins. A person living in this country wants to pay some amount to her neighbor. What is the smallest positive amount that can be paid if both the person and her neighbor have infinitely many coins of each type?

2.2 Optimality

In *optimality* problems we aim to find the maximum possible value of some parameter. To solve such a problem, we should find this optimal value c and prove two claims: (a) c can be achieved (there is a way to get c) and (b) bigger values are impossible (all possible values of the parameter are less than or equal to c). Part (a) is an existential statement (and one example is enough to prove it), while Part (b) is a universal statement (some argument that covers all cases is needed).

2.2.1 Producing Chocolate

Problem 47 — Producing chocolate. A factory produces milk chocolate ($10 profit per box) and dark chocolate ($30 profit per box). The daily demands are 500 and 200 boxes for milk and dark chocolate, respectively. The factory's maximum capacity is 600 boxes of chocolate per day. (This means that the factory can produce any combination of milk and dark chocolate boxes, the only requirement is that the total number of boxes produced is at most 600 per day.) What is the optimal production plan, that is, what is the maximum profit per day?

Stop and Think! Can the factory work at full capacity and still sell all the chocolate produced? What would be the best production plan if we had unlimited demand for both types of chocolate?

The total demand for both types is $500 + 200 = 700$ boxes per day, and this exceeds the total capacity, so we can sell all the chocolate assuming we choose the types correctly (say, 450/150 for milk/dark chocolate). If demand is unlimited, it is more profitable to produce only dark chocolate (30 profit per box instead of 10), i.e., 600 boxes of dark chocolate (more than allowed by the actual demand).

If you have some business experience, at this point you would probably laugh and say: what a stupid problem — of course, one should produce as much of the more profitable dark chocolate as one can sell, that is, 200 boxes of dark chocolate, and use the rest of the production capacity (400 boxes per day) for milk chocolate. This would give a daily profit of $200 \cdot 30 + 400 \cdot 10 = 6000 + 4000 = 10000$. Your intuition would be right. Still, we need some simple examples to work with.

We claim that the maximum profit per day is \$10000 and want to prove it. What does it mean and what should we prove? We need to show two claims:

1. *Existential statement*: there *exists* a production plan achieving a profit of \$10000.
2. *Universal statement*: *all* plans give a profit of at most \$10000, so there is no better one.

To show the first claim, it is enough to show a production plan achieving \$10000: 400 boxes of milk chocolate and 200 boxes of dark chocolate per day. It is indeed a valid plan: no more than 500 boxes of milk chocolate, no more than 200 boxes of dark chocolate, and no more than 600 boxes in total. The resulting profit is

$$\$10 \times 400 + \$30 \times 200 = \$10000.$$

To prove the second claim, we use the following argument that looks strange at first, but is formally correct. Denote by M and D the number of boxes of milk and dark chocolate produced per day. From the problem statement, we know that

$$M \leq 500, \tag{2.1}$$
$$D \leq 200, \tag{2.2}$$
$$M + D \leq 600. \tag{2.3}$$

The profit is equal to

$$10M + 30D.$$

Now, let us multiply the inequality (2.2) by 20 and multiply the inequality (2.3) by 10:

$$\begin{aligned} 20D &\leq 4000, \\ 10M + 10D &\leq 6000. \end{aligned}$$

By summing up these two inequalities, we show that the profit per day is always at most \$10000:

$$10M + 30D \leq 10000.$$

But why on earth did we decide to multiply the second inequality by 20, the third by 10, and then add them? Note also that we completely disregarded the first inequality. This is a good question that can be answered in several ways. First, we may use our mathematicians' right to do whatever we want as long as the reasoning is correct; we do not need to explain how we came up with this reasoning. Second, we can say that there is a special theory, called *linear programming*, that recommends this course of action. This sounds intimidating, but in fact one can show how this theory works in our case using Figure 2.11. Do not worry if this is not clear yet: we just wanted to give an example of optimality proof.

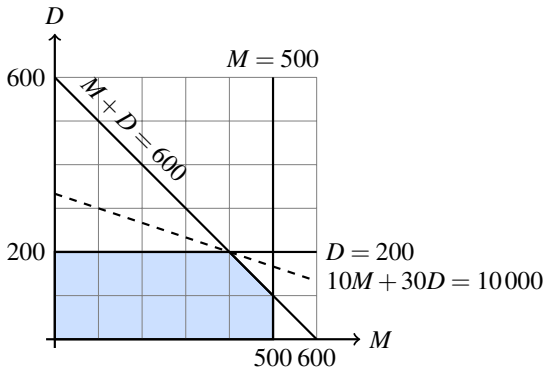


Figure 2.11: We plot all the restrictions on the pair (M, D); the region of possible production plans is highlighted. The dashed line indicates the production plans with a profit of \$10 000 per day. We see that there is exactly one production plan that achieves this profit. The same would be true even in the case of unlimited demand for milk chocolate, so only the inequalities 2.2 and 2.3 matter. We combine them with suitable coefficients to get the dotted line: to have $c_1(M+D)+c_2D = 30D+10M$, we use $c_1 = 10$ and $c_2 = 20$.

Summary

A proof that α is optimal consists of two parts:
- there exists a solution achieving the value α;
- no solutions achieve a value better than α.

2.2.2 Maximum Number of Two-digit Integers

Problem 48 — Maximum number of two-digit integers. There are 90 cards with all two-digit numbers on them:

$$10, 11, 12, \ldots, 98, 99.$$

A player can select some of these cards. For each card chosen, she gets \$1. However, if the player takes two cards that add up to 100 (say, 23 and 77), then she loses all the money. How much money can she gain in this game? Try it: Coursera.

In mathematical language, the problem can be restated as follows: what is the maximum number of elements in a subset of $\{10, 11, \ldots, 99\}$ that does not contain two numbers x and y with $x+y = 100$? An even more formal version is: find

$$\max\{|S|\colon S \subseteq \{10, \ldots, 99\} \text{ and } \forall x \neq y \in S,\ (x+y \neq 100)\}.$$

Don't be frightened by the notation: we will learn it later step by step.

Stop and Think! Why do we have exactly 90 two-digit numbers?

One could count them by tens: there are ten numbers that start with 1 (i.e., $10, 11, 12, \ldots, 18, 19$), ten numbers starting with 2, ..., ten numbers starting with 9. Another way to count: there are 99 numbers from 1 to 99, we delete 9 numbers from 1 to 9, and we get $99 - 9 = 90$. In general, the range $m \ldots n$ (for integer $m \leq n$) contains $n - (m-1) = n - m + 1$ numbers. (A common mistake here is to forget the "+1" correction.)

Let us return to the original problem. There are pairs of numbers that sum up to 100, e.g., $10 + 90 = 100$. If the player takes 10, then she cannot take 90, and vice versa. Similarly, 11 and 89 cannot be taken simultaneously. In total, we have 40 such pairs:

$$(10, 90), (11, 89), \ldots, (49, 51). \tag{2.4}$$

Stop and Think! What two-digit numbers are left without pairs?

The integers $91, 92, \ldots, 99$ do not have pairs. Are there any others? Yes, there is one more number without a pair, 50. In total, we have ten numbers without pairs:

$$50, 91, 92, \ldots, 99.$$

Hence, one definitely cannot take more than $40 + 10 = 50$ numbers (one number from each pair, and all the unpaired numbers).

Stop and Think! Can you find a solution with exactly 50 numbers?

There are many ways to provide such an example (we are free to choose one of the elements from each pair). Suppose we select the larger element in each pair. Then we get

$$50, 51, \ldots, 99.$$

Let us repeat this argument in a more concise and formal way.

Theorem 2.2.1 The maximum number of two-digit integers such that no two of them sum up to 100 is 50.

Proof. Denote this maximum by M. Below, we show that $M \geq 50$ and $M \leq 50$. This means that $M = 50$.

Existential part. There *exists* such a set of size 50: $50, 51, \ldots, 99$. Indeed, the sum of any two numbers from this set is greater than 100. This shows that $M \geq 50$.

Universal part. On the other hand, *any* such set must exclude at least one number from each of the 40 pairs from (2.4). Hence, the size of the set is at most $90 - 40 = 50$. This implies that $M \leq 50$. ■

2.2.3 Rooks on a Chessboard

A chess *rook* moves vertically and horizontally; see Figure 2.12. Chess players say that the rook *attacks* all cells to which it can move, i.e., all cells on the same vertical or horizontal line.

Stop and Think! How many cells does a rook attack (not including its own cell)?

Each vertical (or horizontal) line contains 8 cells, so there are 7 attacked cells (in addition to the rook cell), for a total of 14 cells.

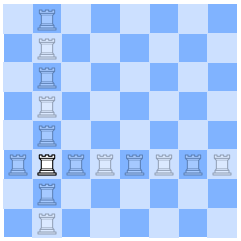


Figure 2.12: A chess rook moves vertically and horizontally.

Problem 49 What is the maximum number of rooks on a chessboard such that no two rooks attack each other? Try it: Coursera.

To answer this problem, let us reformulate the rules: two rooks attack each other if they are in the same row (horizontal line) or column (vertical line).

Stop and Think! Do you see why the number of rooks not attacking each other is at most 8?

The reason is obvious: each of 8 columns may contain at most one rook, so there are at most 8 rooks.

Stop and Think! In this argument, we talked about columns and did not say anything about rows. Is that OK?

Yes — it shows that the restriction "no two rooks in one column" is enough to prove that there are at most 8 rooks.

Stop and Think! Can we say now that the problem is solved and we proved that the maximum number of rooks not attacking each other is 8?

Not yet, but we are close. It remains to show the first (existential) part of the proof: we need to provide an example with 8 rooks not attacking each other. There are many ways to do this, see Figure 2.13.

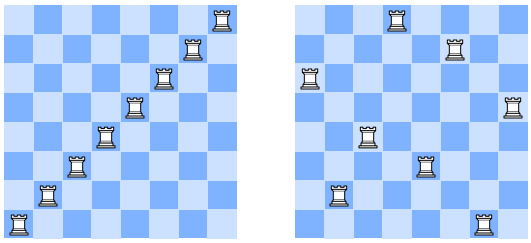


Figure 2.13: Two ways of placing eight rooks that do not attack each other. (Do we really need both of them? Of course not, one would be enough.)

We have seen two possible solutions to the non-attacking rooks problem. There are many others: can you say how many? Later in the book, we will learn a way to compute this number.

Problem 50 Imagine a "chessboard" with 10 columns and 15 rows. What is the maximum number of rooks not attacking each other on this board?

2.2.4 Knights on a Chessboard

Consider another chess piece: a *knight*. It moves two cells in one direction (vertical or horizontal) and one in the other (perpendicular) direction, see Figure 2.14.

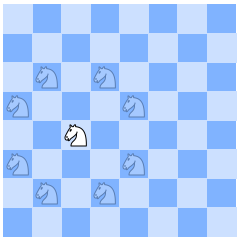


Figure 2.14: A chess knight makes an L-shaped move.

Stop and Think! What is the maximum number of cells that a knight may attack?

The maximum number is 8 if the knight is far from the border. Some of the moves become impossible if the knight is close to the border. For example, if we move the knight one cell to the left, two of the moves become impossible and we get 6 attacked cells.

Problem 51 List all the possible numbers of cells that a knight may attack.

You have probably guessed what our next question is.

Problem 52 What is the maximum number of knights on a chessboard such that no two of them attack each other? Try it: Coursera.

This is a more difficult problem, and it is instructive to think about it before reading further. But the crucial observation is again related to the dark and light coloring of the chessboard: *a knight never attacks cells of the same color as the cell it occupies*.

Stop and Think! What are the consequences of this observation for our goal (maximum number of knights)?

Let us reformulate this observation: *two knights placed on cells of the same color cannot attack each other*. Therefore — aha! — we can safely fill all light cells (or, if you prefer, all dark cells) with knights (Figure 2.15).

Stop and Think! How many knights did we place this way?

The number of knights, that is, the number of light cells, is 32 (half of the total number of cells; the board contains equal numbers of light and dark cells). We cannot add any more knights, since every remaining cell (every dark cell) is under attack by one or more knights. Thus, the maximum number of knights is 32 and the problem is solved.

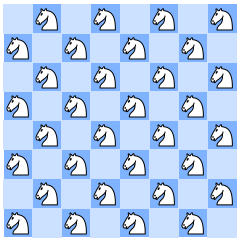


Figure 2.15: Each dark cell has a knight.

Stop and Think! Do you see a mistake in this reasoning?

This mistake is often overlooked by beginners. We proved that we cannot *add* any knights to this configuration, and that is correct. But maybe we can start anew and place knights differently, using both light and dark cells to place more knights — why not? Imagine, for example, that we have filled the third and sixth columns with knights (Figure 2.16). No more knights can be added — still, as we have seen, 16 is far from the maximum.

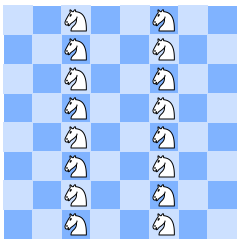


Figure 2.16: In this configuration, we have 16 knights and can't add more since all the cells are under attack. But this does not mean that 16 is the maximum number of knights on the board.

Similarly, there is no reason to be sure that 32 is the maximum number of knights. We need to prove that we cannot place more than 32 knights, and we do not have that yet.

Stop and Think! Do you see such a proof? (Recall the problem with integers not summing up to 100: a similar argument may work, too.)

In Problem 48, we grouped the numbers in pairs in such a way that one cannot take both numbers from a pair. We can pair the cells similarly. If two cells differ by a knight move, they cannot both be used (Figure 2.17).

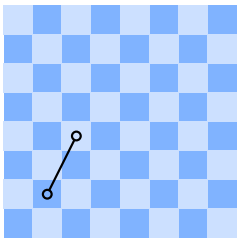


Figure 2.17: A pair of cells that can't be used together.

Thus, if we can group all 64 cells into 32 pairs connected by a knight move, then each pair could contain only one knight, and therefore the number of knights is at most 32.

Stop and Think! Can you construct such a pairing?

We can start with a smaller board of size 2×4 (boards smaller than this do not work).

Stop and Think! Do you see how to group the 8 cells of a 2×4 board into 4 pairs so that the cells in each pair are connected by a knight move? Do you see how to use this pairing to construct a pairing for an 8×8 board?

The required pairing for a 2×4 board is shown in Figure 2.18.

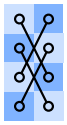


Figure 2.18: A 2×4 board: four pairs.

We can combine eight small boards to get a big one (Figure 2.19). Since only one cell from each pair can be used, this picture proves that we can't place more than 32 knights and thus finishes the proof.

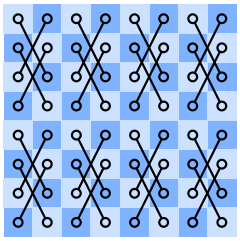


Figure 2.19: Combining eight 2×4 boards to get a pairing for 8×8 board.

In this problem, we saw a difference between an optimal configuration with the maximum number of knights (e.g., Figure 2.15) and a suboptimal configuration where no knights can be added (e.g., Figure 2.16).

2.2.5 Bishops on a Chessboard

A *bishop* attacks diagonally in every direction (Figure 2.20).

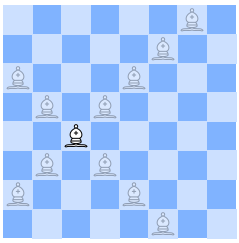
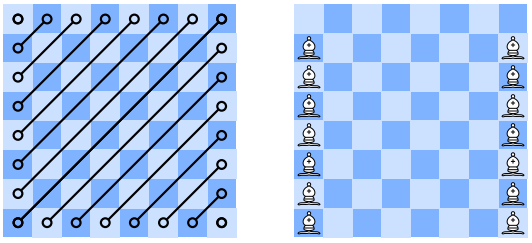


Figure 2.20: A bishop moves diagonally.

Problem 53 How many bishops not attacking each other can be placed on a chessboard? Try it: Coursera.

Now we have enough experience to present a solution to this problem in a rigorous but concise form. Each of the 15 diagonals (including the two trivial "diagonals", the top left corner and the bottom right corner) shown in Figure 2.21(a) may contain only one bishop. In addition, only one of two trivial diagonals can be used since they are connected by a bishop's move. Thus, the maximum number of bishops on a board is at most 14. On the other hand, one can place 14 bishops not attacking each other, see Figure 2.21(b).



(a) (b)

Figure 2.21: A solution to Problem 53.

There are two remaining chess pieces, the royal ones. If you know how they move, you can try to determine the maximum number of *queens* that can be placed on a chessboard. Since the rook's moves are available to queens, we cannot place more than eight queens; it turns out (see Section 2.3) that it is indeed possible to put eight queens on a chessboard. Kings are less aggressive than queens (they can move only to one of the 8 neighboring cells), and for them, the problem is easier.

Problem 54 Prove that one can place 16 kings not attacking each other, but no more.

Hint: divide the board into 2×2 squares.

There is another type of interesting question: what is the *minimum* number of kings that suffices to attack all the cells (that are not occupied by kings). It is also an optimization problem (here "optimal" means "minimum" rather than "maximum"). To solve it, you should specify some value n and

- show an example with n kings attacking all the remaining cells;
- prove that fewer than n kings are not enough (i.e., there exists a non-attacked cell without a king in it).

Problem 55 Prove that the minimum number of kings attacking all cells is $n = 9$.

Hint: cut the board into 3×3 pieces (this can be done for a 9×9 board, but for an 8×8 board some pieces will have to be smaller). Show that each piece can be "covered" by one king, and that one king should be present in each piece.

In the language of graph theory, this problem is called the *dominating set* problem, whereas the previous problems were examples of the *independent set* problem.

2.2.6 Subsets without x and $2x$

In Problem 48, we were asked to select the maximum number of two-digit numbers such that no two of them sum up to 100. Now, let us consider a similar problem but with another restriction.

Problem 56 What is the maximum number of integers among $1, 2, \ldots, 50$ that one can select if one is not allowed to select x and $2x$ simultaneously? Try it: Coursera.

Stop and Think! Can we again split the numbers into pairs of mutually incompatible numbers that can't be selected together?

Not exactly. We can't select 1 and 2 simultaneously, so they should form a pair. But we also can't select 2 and 4 simultaneously, so 2 should also be paired with 4. Then 4 should be paired with 8, and 8 should be paired with 16, etc. This way, we get a chain

$$1 - 2 - 4 - 8 - 16 - 32,$$

where the neighbors can't be used simultaneously. (The next number, 64, is outside our range.) In general, x cannot appear with $x/2$ or $2x$ (if present among $1, 2, \ldots, 50$).

Stop and Think! Can you list other pairs of numbers that can't be selected together?

The answer can be presented graphically (Figure 2.22).

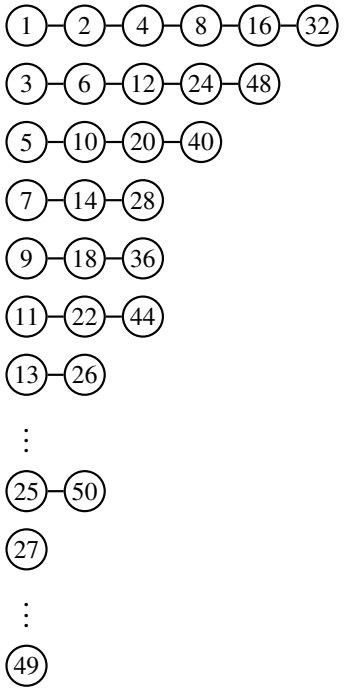


Figure 2.22: Chains of mutually incompatible numbers.

Each node contains a number between 1 and 50. Mutually incompatible numbers are connected by edges. Each chain starts with some odd x (if x is even, then it follows $x/2$ in a different chain) and continues with $2x$, $4x$, etc. as long as possible (numbers do not exceed 50). For numbers $27, 29, \ldots, 49$ the chains are trivial (since there is no node with $x/2$ or $2x$).

Stop and Think! How can we now reformulate our question in terms of the graph in Figure 2.22?

The integers $1, 2, \ldots, 50$ are nodes of the graph, and the restriction is that we should not select two nodes connected by an edge.

Stop and Think! What is the maximum number of nodes that can be selected according to this rule?

The crucial observation here is that the *choices in different chains are independent* (there are no edges between chains). Hence, we can choose the maximum number of nodes in each chain separately and then combine our choices! In the first chain, we can choose three nodes (say, $1, 4, 16$, or $2, 8, 32$, or $1, 8, 32$, etc.). In the second chain, we can also select three nodes, but there is only one way to do so: $3, 12, 48$. Then we have chains with two nodes, starting with $5, 7, 9, 11$, and all the remaining chains give only one node. In total, we get the sum $3+3+2+2+2+2+1+1+\cdots+1$.

Stop and Think! How many terms do we have in this sum? How many of them are equal to 1?

Each chain gives us one term in the sum, so the number of terms is the number of chains. The chains start with all odd numbers from this range: $1, 3, 5, \ldots, 49$. Since each odd number can be paired with the following even number, the number of odd numbers (and the number of chains) is $50/2 = 25$. There are 6 terms greater than 1 (recall $3+3+2+2+2+2$), and therefore there are $25-6 = 19$ terms that are equal to 1. Then the total sum is $3+3+2+2+2+2+19 = 33$. Our reasoning therefore shows that the maximum number of integers that can be selected according to the rules is 33.

Stop and Think! Can you write an example of 33 numbers that can be selected according to these rules?

As we have discussed, there are many ways to do this. One is to take all the numbers in the first, third, and fifth columns. This gives $25+6+2 = 33$ numbers.

Problem 57 How many different maximum solutions are there (subsets of $\{1, \ldots, 50\}$ of size 33 that do not include x and $2x$ at the same time)?

We'll see how to do this using the product rule later in the book. However, if you're curious, the answer is 1 536!

Problem 58 What is the maximum number of distinct integers from $1, 2, \ldots, n$ that can be selected if we are not allowed to select x and $2x$ at the same time? In the previous problem, we had $n = 50$.

The answer to this problem is a bit more complicated:

$$\left\lfloor \frac{n+1}{2} \right\rfloor + \left\lfloor \frac{n/4+1}{2} \right\rfloor + \left\lfloor \frac{n/4^2+1}{2} \right\rfloor + \ldots,$$

where $\lfloor u \rfloor$ stands for the integer part of u, also known as the *floor* function (the maximum integer not exceeding u), and the sum is taken until the terms become zero. For $n = 50$, we get $25+6+2+0+0+\ldots = 33$ (the answer that we already know). Try to prove this formula for all values of n. One way to prove this is to show that the first column has $\left\lfloor \frac{n+1}{2} \right\rfloor$ numbers, the third column has $\left\lfloor \frac{n/4+1}{2} \right\rfloor$ numbers, etc.

2.3 Computer Search

This section discusses more advanced techniques for using computers to construct tricky combinatorial objects. If you have never written recursive programs before, we encourage you to skip this section for now and return here after you learn recursion in Section 3.1.

We have already seen examples of problems in which a solution exists but is not at all easy to construct by hand. A natural thing to do in such a case is to ask a machine to do the job. But if the search space is enormous, then one still needs to help the machine to narrow the search space. In this section, we will practice implementing efficient programs for this task. As usual, to introduce the underlying techniques, we use puzzles with simple rules (so that no problem-specific details distract you from understanding the main ideas). Still, the technique that we introduce, *backtracking*, finds applications in a wide range of real-life optimization problems — planning, scheduling, supply chain optimization, and logistics, to name a few.

Our working example for introducing the *backtracking* technique will be the classical n queens problem. Recall that a *chess queen* moves vertically, horizontally, or diagonally (see Figure 2.23).

Problem 59 — n queens. Is it possible to place n queens on an $n \times n$ board such that no two of them attack each other? Try it: Coursera.

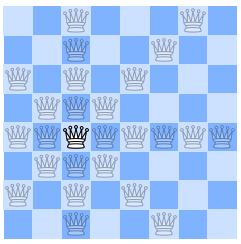


Figure 2.23: Moves of a chess queen.

It is not that difficult to find the answer for $n \leq 5$ (try it!), but already for $n = 8$ it is not easy to construct a solution by hand.

It is known that, for any $n \geq 4$, the answer is positive: one can place n non-attacking queens on an $n \times n$ board for any integer $n \geq 4$. Moreover, there is an *explicit* solution to this task: there is a simple formula specifying where to place the queens (the fact that the formula is simple means that it is easy to *state it* and does *not* mean that it is easy to *find it*!).

Problem 60 Assume that $n \geq 4$ is even and the remainder of n when divided by 6 is not equal to 2. Prove that the following placement is correct (i.e., no two queens attack each other): for every $0 \leq i \leq n/2-1$, place two queens on the cells $(i, 2i+1)$ and $(n/2+i, 2i)$. An example of this placement for $n = 12$ is shown in Figure 2.24.

This means that one does not need a computer to solve this problem. Still, the n queens problem is a popular benchmark for various combinatorial optimization programs. Implementing such a program is also an interesting and instructive problem in algorithms. For this reason, in the rest of this section, we will be solving the following problem.

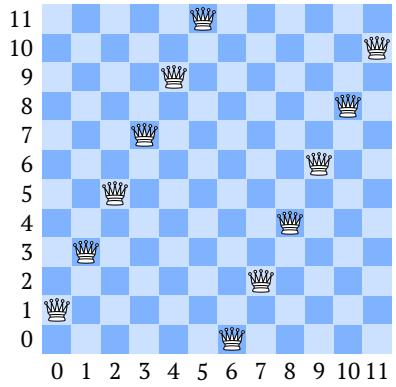


Figure 2.24: A correct placement of 12 queens according to the formula from Problem 60.

Problem 61 Implement a program that, given $n \geq 4$, finds a correct placement of n queens on an $n \times n$ board.

2.3.1 Brute Force

We start by implementing a brute force solution. Specifically, we will carry out the following two steps.

Step 1. Enumerate all possible placements of n queens in which no two queens lie in the same row or column.

Step 2. Among all such placements, select one in which no two queens lie on the same diagonal.

Stop and Think! Consider a placement of n queens no two of which lie in the same row or column. How do we represent such a placement in a program?

To represent it, we may use a sequence $r_0, r_1, \ldots, r_{n-1}$ of n integers: r_i is the index of the row containing a queen in the i-th column. That is, a sequence $r_0, r_1, \ldots, r_{n-1}$ defines the following n cells on the board:

$$(0, r_0), (1, r_1), \ldots, (n-1, r_{n-1}).$$

Since we know that no two queens lie in the same row, all r_i's must be different. This, in turn, means that $r_0, r_1, \ldots, r_{n-1}$ is nothing else but a *permutation* of the numbers $0, 1, \ldots, n-1$. In Figure 2.25, we give examples of such permutations.

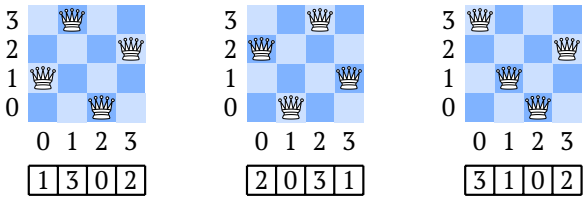


Figure 2.25: A placement of n queens in different rows and columns can be represented as a permutation.

A permutation is a fundamental object in discrete mathematics. Therefore, it is not at all surprising that `Python` has built-in methods for enumerating permutations. Using the `permutations()` function from the `itertools` library we implement step 1 in just one line of code!

```
from itertools import permutations

for permutation in permutations(range(4)):
    print(permutation)
```

```
(0, 1, 2, 3)
(0, 1, 3, 2)
(0, 2, 1, 3)
(0, 2, 3, 1)
(0, 3, 1, 2)
(0, 3, 2, 1)
(1, 0, 2, 3)
(1, 0, 3, 2)
(1, 2, 0, 3)
(1, 2, 3, 0)
(1, 3, 0, 2)
(1, 3, 2, 0)
(2, 0, 1, 3)
(2, 0, 3, 1)
(2, 1, 0, 3)
(2, 1, 3, 0)
(2, 3, 0, 1)
(2, 3, 1, 0)
(3, 0, 1, 2)
(3, 0, 2, 1)
(3, 1, 0, 2)
(3, 1, 2, 0)
(3, 2, 0, 1)
(3, 2, 1, 0)
```

We proceed to step 2. Looking at Figure 2.25, we see that not every permutation constitutes a solution to the n queens problem. For example, the first two permutations in the figure are valid solutions, whereas the last one is not, as two queens attack each other.

Stop and Think! How do we check whether a permutation contains two queens on the same diagonal?

To do this, we need to check whether two cells (i_1, j_1) and (i_2, j_2) lie on the same diagonal. This happens if and only if

$$|i_1 - i_2| = |j_1 - j_2|,$$

i.e., if the horizontal shift is the same as the vertical shift (see Figure 2.26).

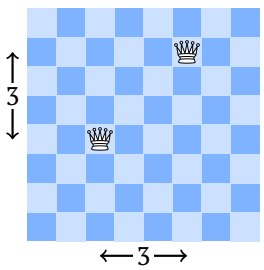


Figure 2.26: Two queens are on the same diagonal.

This observation allows us to implement the following simple procedure for checking whether the given permutation gives a correct solution.

```
from itertools import combinations


def is_solution(perm):
    pairs = combinations(range(len(perm)), 2)
    return all(abs(i1 - i2) != abs(perm[i1] - perm[i2])
               for i1, i2 in pairs)


print(is_solution([1, 3, 0, 2]))
print(is_solution([2, 0, 3, 1]))
print(is_solution([3, 1, 0, 2]))
```

```
True
True
False
```

In this code, we use another useful function from the `itertools` library: `combinations` allows us to enumerate all pairs (i_1, i_2) such that $0 \le i_1 < i_2 \le n-1$.

Finally, by combining the two steps that we've implemented, we get a program for the n queens problem (basically, just six lines of code!). In the code, the function `filter` is used to disregard all permutations for which the function `is_solution` returns `False`. The function `next` just returns the first permutation satisfying our property. This first permutation is shown in Figure 2.27.

```
from itertools import combinations, permutations


def is_solution(perm):
    return all(abs(i1 - i2) != abs(perm[i1] - perm[i2])
               for i1, i2 in combinations(range(len(perm)), 2))


print(next(filter(is_solution, permutations(range(8)))))
```

```
(0, 4, 7, 5, 2, 6, 1, 3)
```

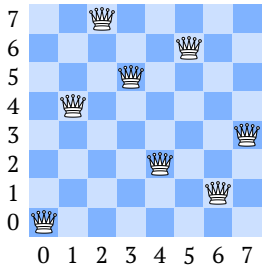


Figure 2.27: A solution to the 8 queens problem found by the program that we've implemented.

The resulting program is simple and short. At the same time, it is too slow. Already for $n = 13$, it starts taking a noticeable amount of time to complete. The reason is that it is not at all optimized: it just goes through all permutations and finds the first one that gives a correct solution. For this reason, it is called a *brute force* solution. Below, we show how to optimize it so that it works fast enough even for $n = 20$.

2.3.2 Backtracking

The main idea of the powerful *backtracking* method is the following:

- We construct a solution piece by piece: first, we place a queen in the first column; then, we place a queen in the second column, and so on.
- At each point, we check whether the given *partial solution* is correct: for example, if the queens in the first two columns already lie on the same diagonal, there is no point in extending the current partial solution further. In such a case, we *backtrack*.

This is best illustrated by an example. Figure 2.28 shows how one would try to place three queens on a 3×3 board. We start with an empty board and try to place a queen in the leftmost column. We first place a queen in the bottom row. This leaves a single non-attacked cell in the middle column (all attacked cells on every board are crossed out). If one places a queen in this cell, then all cells in the rightmost column are attacked. At this point, we realize that we should backtrack and try to place a queen in the leftmost column in some other cell. If we use the middle cell, then all cells in the middle column are attacked, so we discard this partial solution and backtrack again. Finally, we try the top row in the leftmost column. This branch leads to a dead end similar to the one in the leftmost branch of the tree.

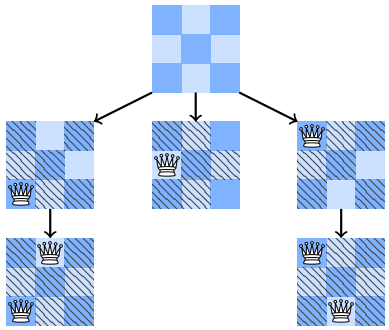


Figure 2.28: A backtracking approach to the 3 queens problem.

To implement the backtracking approach, we need to be able to enumerate partial permutations. This can be done as follows: given a partial permutation, extend it with an element that it does not contain and continue recursively.

```
def extend(perm, n):
    if len(perm) == n:
        print(perm)
        return

    for k in range(n):
        if k not in perm:
            extend(perm + [k], n)


extend(perm=[], n=3)
```

```
[0, 1, 2]
[0, 2, 1]
[1, 0, 2]
[1, 2, 0]
[2, 0, 1]
[2, 1, 0]
```

As usual, it is instructive to add a few debug printing instructions to better understand the behavior of the code.

```
def extend(perm, n):
    if len(perm) == n:
        print(f'Final permutation: {perm}')
        return

    print(f'Extending partial permutation {perm}...')
    for k in range(n):
        if k not in perm:
            extend(perm + [k], n)


extend(perm=[], n=3)
```

```
Extending partial permutation []...
Extending partial permutation [0]...
Extending partial permutation [0, 1]...
Final permutation: [0, 1, 2]
Extending partial permutation [0, 2]...
Final permutation: [0, 2, 1]
Extending partial permutation [1]...
Extending partial permutation [1, 0]...
Final permutation: [1, 0, 2]
Extending partial permutation [1, 2]...
Final permutation: [1, 2, 0]
Extending partial permutation [2]...
Extending partial permutation [2, 0]...
Final permutation: [2, 0, 1]
Extending partial permutation [2, 1]...
Final permutation: [2, 1, 0]
```

OK, we now know how to generate permutations recursively. We do this by starting from an empty permutation and extending it gradually. The next step is to detect partial permutations that cannot be extended to a solution to the n queens problem. To do this, we just check whether the last element of the permutation (i.e., the rightmost queen) conflicts with any of the previous elements (i.e., is attacked by one of the previous queens).

```
def can_be_extended(perm):
    i = len(perm) - 1
    return all(i - j != abs(perm[i] - perm[j])
               for j in range(i))
```

Finally, we have everything we need to write our backtracking program! For $n = 16$, it finds a solution immediately.

```
def can_be_extended(perm):
    i = len(perm) - 1
    return all(i - j != abs(perm[i] - perm[j])
               for j in range(i))


def extend(perm, n):
    if len(perm) == n:
        print(perm)
        exit()
```

```
    for k in range(n):
        if k not in perm:
            if can_be_extended(perm + [k]):
                extend(perm + [k], n)


extend(perm=[], n=16)
```

```
[0, 2, 4, 1, 12, 8, 13, 11, 14, 5, 15, 6, 3, 10, 7, 9]
```

In Section 7.2, we show how to implement a program that works in the blink of an eye even for $n = 100$.

To practice implementing backtracking solutions, try to solve the following problem (it is not easy to solve by hand!).

Problem 62 In a 5×5 grid, draw 16 diagonals that do not touch each other. (E.g., in 3×3 and 4×4 grids one can place 6 and 10 diagonals, respectively. See Figure 2.29.) Try it: Coursera.

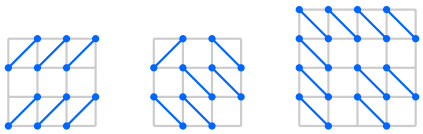


Figure 2.29: Placing many non-intersecting diagonals.

A backtracking approach for this problem may proceed as follows:

- Fill in the grid gradually (say, from bottom to top, from left to right).
- For each cell consider three possibilities:
  1. Diagonal from bottom left corner to top right corner.
  2. Diagonal from bottom right corner to top left corner.
  3. No diagonal.
- Whenever a new diagonal is placed, check whether it conflicts with other diagonals. If it does, backtrack.

We encourage you to post your solution and to check solutions submitted by other learners in the forum thread.

Problem 63 Prove that one cannot place more than 16 non-intersecting diagonals on a 5×5 grid.

This is another challenging problem. To see that 16 is optimal, note that a 5×5 grid contains 36 nodes and each diagonal must use two of them. This already means that it is impossible to place more than 18 diagonals. To see that placing even 17 diagonals is impossible, see Figure 2.30. It focuses our attention on two layers of nodes: the outer one (shown in blue) and the layer next to it (shown in black). If we manage to place 17 diagonals, then only two nodes will be unused.

Every blue node can be joined by a diagonal to either another blue node or a black node. At the same time, if we join two blue nodes (this is only possible for two blue nodes that are close to the corner), we block the corner blue node. Thus, even if we join every black node with some blue node, we will still have eight remaining blue nodes that can be joined only to each other. To leave only two of them unused, we need to join at least three pairs of them, but (as discussed above) this makes three blue nodes unused.

When you implement a backtracking program for the 5×5 grid and make sure that it finds an answer, you might want to check the behavior of your program on $n \times n$ grids for $n > 5$ (and

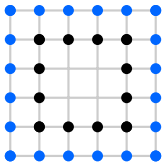


Figure 2.30: Two layers of nodes in the 5×5 grid.

even for $n \times m$ grids). Interestingly, already for $n = 27$ the exact value of the maximum number of diagonals for the $n \times n$ grid is not currently known! See the corresponding sequence in the Online Encyclopedia of Integer Sequences.

Another way to generalize the diagonals problem is to consider diagonals of length $\sqrt{5}$ in 1×2 rectangles. For example, one can place 20 such diagonals on a 6×6 grid, see Figure 2.31.

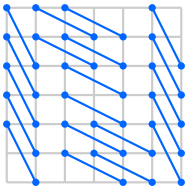


Figure 2.31: 20 non-intersecting 1×2 diagonals on a 6×6 grid.

Problem 64 Prove that it is possible to place 21 non-intersecting diagonals of length $\sqrt{5}$ in a 6×6 grid.

In Section 7.3, we state the diagonals problems as Integer Linear Programming (ILP) problems and use ILP solvers to solve them efficiently even when n is as large as 15.

Summary

- In some situations, an object exists but is not easy to construct by hand. In this case, one may want to use a computer to find such an object.
- If the search space is small enough, a naive brute force search might help to find the required object. However, in many cases, the search space is enormous and one should use additional techniques to narrow the search space.
- One such technique is known as backtracking. Its main idea is to construct the required object step by step and to take a step back as soon as a conflict is detected.